

12 复合模式

模式的模式



谁能料到模式居然可以携手合作？你已经见识过炉边谈话的火爆场面（你们还没见识过模式死亡匹配的篇章呢，编辑逼着我们移除了）。谁能料到模式居然可以携手合作？好吧，信不信由你，有一些威力最强大的OO设计同时使用数个设计模式。准备好升级你的模式技巧，现在是复合模式时间。

携手合作

模式的一种最棒的用法，就是把它们从家里拿出来，和其他模式交互。你模式用得越多，越容易发现它们一同现身在你的设计中。对于这些在设计中携手合作来征服许多问题的模式，我们有一个特别的名字：复合模式（Compound Pattern）。没错，我们正在谈论一种由模式构成的模式。

在现实世界中，你会用到许多复合模式。现在你的脑袋中已经有许多模式，你会发现复合模式其实只是模式携手合作，这样理解就容易多了。

本章，我们从重访SimUDuck鸭子模拟器中那些熟悉的鸭子开始。当我们把模式结合起来时，使用鸭子的例子是非常恰当的，毕竟，在整本书中，鸭子一直与我们同在，参与了许多模式。鸭子帮助你理解模式在同一个解决方案中如何携手合作。我们将某些模式结合使用，并不意味着这个解决方案有资格称为复合模式。复合模式必须是普适目标的方案，能应用于解决许多问题。因此，在本章的后半段，我们会造访一个真正的复合模式：模型-视图-控制器（Model-View-Controller），或者称为MVC。如果你没听过MVC，那你会发现，这会是你的设计工具箱内最强大的模式之一。



模式通常被一起使用，并结合在同一个设计解决方案中。

复合模式在一个解决方案中结合两个或多个模式，以解决一般的或重复发生的问题。

与鸭子重聚

正如你已经知道的，我们会再次和鸭子合作，这一次，鸭子将在同一个解决方案中，展示模式如何共存甚至合作。

我们将从头重建我们的鸭子模拟器，并通过使用一堆模式，给它一些有趣的能力，OK，我们开始……

① 首先，我们将创建一个Quackable接口。

就像之前说的，我们将从头开始。而这一次，鸭子将实现Quackable接口，这样，我们就知道模拟器中，有哪些东西可以呱呱叫，像绿头鸭、红头鸭。甚至可能还会看到橡皮鸭偷偷溜回来。

```
public interface Quackable {
    public void quack();
}
```

Quackables只需要做好一件事：呱呱叫！

② 现在，某些鸭子实现Quackable。

如果没有一些类来实现某个接口，那么接口就没有意义。现在就来创建一些具体的鸭子（不是那种“玩偶鸭”，你知道我们指的是什么）。

```
public class MallardDuck implements Quackable {
    public void quack() {
        System.out.println("Quack");
    }
}
```

标准的绿头鸭。

```
public class RedheadDuck implements Quackable {
    public void quack() {
        System.out.println("Quack");
    }
}
```

如果我们希望这个模拟器变得有趣，就要有一些物种变体。

如果我们没有添加别的种类的鸭子，就不太好玩。

还记得上次吧？我们曾经加入鸭鸣器（猎人用的那种东西，它们肯定会呱呱叫）以及橡皮鸭。

```
public class DuckCall implements Quackable {
    public void quack() {
        System.out.println("Kwak");
    }
}

public class RubberDuck implements Quackable {
    public void quack() {
        System.out.println("Squeak");
    }
}
```

DuckCall（鸭鸣器）会呱呱叫，但听起来并不十分像真的鸭叫。

RubberDuck（橡皮鸭）呱呱叫时，其实是吱吱叫。

③ 好了，我们有了鸭子，还需要一个模拟器。

让我们制造一个模拟器，这个模拟器会创建一些鸭子，并保证它们会呱呱叫……

```
public class DuckSimulator {
    public static void main(String[] args) {
        DuckSimulator simulator = new DuckSimulator();
        simulator.simulate();
    }

    void simulate() {
        Quackable mallardDuck = new MallardDuck();
        Quackable redheadDuck = new RedheadDuck();
        Quackable duckCall = new DuckCall();
        Quackable rubberDuck = new RubberDuck();

        System.out.println("\nDuck Simulator");

        simulate(mallardDuck);
        simulate(redheadDuck);
        simulate(duckCall);
        simulate(rubberDuck);
    }

    void simulate(Quackable duck) {
        duck.quack();
    }
}
```

我们的main()方法将让所有的事情运作起来。

我们创建一个模拟器，然后调用其simulate()方法。

我们需要一些鸭子，所以在这里把每个Quackable创建一个实例……

……然后模拟每种鸭子。

在这里，我们重载simulate()方法来模拟一只鸭子。

这里，我们让多态施展它的魔法：不管传入的是哪一种Quackable对象，simulate()方法都让它呱呱叫。

不要太兴奋，我们还没加上模式呢！



File Edit Window Help ItBetterGetBetterThanThis

```
% java DuckSimulator
Duck Simulator
Quack
Quack
Kwak
Squeak
```

它们都实现同一个Quackable接口，
允许以各自的方式呱呱叫。

到目前为止一切似乎还行。

④ 当鸭子出现在这里时，鹅也离得不远。

只要有水塘的地方，就可能会有鸭和鹅。我们搞一个Goose（鹅）类放在模拟器附近。

```
public class Goose {
    public void honk() {
        System.out.println("Honk");
    }
}
```



鹅的叫声是咯咯，而不是呱呱。



假设我们想要在所有使用鸭的地方使用鹅，毕竟鹅也会叫、会飞、会游。为什么我们不能在模拟器中使用鹅？

什么模式可以让我们轻易地将鸭和鹅掺合在一起？

⑤ 我们需要一个鹅适配器。

我们的模拟器期望看到Quackable接口，既然鹅不会呱呱叫（它们属于咯咯派），我们可以用适配器将鹅适配成鸭。

```
public class GooseAdapter implements Quackable {
    Goose goose;

    public GooseAdapter(Goose goose) {
        this.goose = goose;
    }

    public void quack() {
        goose.honk();
    }
}
```

记住，适配器实现目标接口，
本例即Quackable。

← 构造器需要传入要适配的鹅。

← 当调用quack()时，会被委托到鹅的honk()方法。

⑥ 现在模拟器也应该可以使用鹅了。

我们需要创建一个Goose对象，将它包装进实现Quackable的适配器，这样，我们就可以继续了。

```
public class DuckSimulator {
    public static void main(String[] args) {
        DuckSimulator simulator = new DuckSimulator();
        simulator.simulate();
    }
}
```

```
void simulate() {
    Quackable mallardDuck = new MallardDuck();
    Quackable redheadDuck = new RedheadDuck();
    Quackable duckCall = new DuckCall();
    Quackable rubberDuck = new RubberDuck();
    Quackable gooseDuck = new GooseAdapter(new Goose());
}
```

通过把Goose包装进
GooseAdapter，就可以让鹅
的行为像鸭子一样。

```
System.out.println("\nDuck Simulator: With Goose Adapter");
```

```
simulate(mallardDuck);
simulate(redheadDuck);
simulate(duckCall);
simulate(rubberDuck);
simulate(gooseDuck);
}
```

一旦鹅被包装起来，我们就可以像其他的
鸭子Quackable对象那样对待它。

```
void simulate(Quackable duck) {
    duck.quack();
}
}
```

⑦ 现在我们来快速运行一下……

这一次运行模拟器时，传递给simulate()方法的对象列表包括一个包裹在鸭子适配器中的Goose。结果？应该会出现咯咯叫（honk）！

```
File Edit Window Help GoldenEggs
% java DuckSimulator
Duck Simulator: With Goose Adapter
Quack
Quack
Kwak
Squeak
Honk
```

有鹅了！现在鹅可以和其他鸭子一样叫出声。



呱呱叫学

呱呱叫学家为可呱呱叫行为的所有方面着迷，方法学家经常研究的其中一件事是：在一群鸭子中，总共发出了多少呱呱叫声？

在不必修改鸭子类的情况下，我们如何添加统计嘎嘎叫次数的能力？

你能想出什么模式可以帮上忙吗？

J. Brewer,
公园巡视者和
呱呱叫学家



⑧ 我们会让这些呱呱叫学家满意，给他们叫声的统计。

怎么做？让我们创建一个装饰者，通过把鸭子包装进装饰者对象，给鸭子一些新行为（计算次数的行为）。我们根本不必修改鸭子的代码。

QuackCounter是一个装饰者。

和适配器一样，我们需要实现目标接口。

我们用一个实例变量来记录被装饰的呱呱叫者。

要统计所有呱呱叫，我们用静态变量跟踪。

```
public class QuackCounter implements Quackable {
    Quackable duck;
    static int numberOfQuacks;

    public QuackCounter (Quackable duck) {
        this.duck = duck;
    }

    public void quack() {
        duck.quack();
        numberOfQuacks++;
    }

    public static int getQuacks() {
        return numberOfQuacks;
    }
}
```

取得在构造器中装饰的Quackable引用。

当quack()被调用时，我们把调用委托给正在装饰的Quackable对象……

……然后把叫声的数量加一。

给装饰者添加一个静态方法，这个方法返回在所有Quackable中发生的叫声次数。

⑨ 我们需要更新模拟器，以创建被装饰的鸭子。

现在，我们必须包装在QuackCounter装饰者中实例化的每个Quackable对象，要是不这么做，鸭子就会到处乱跑，无法统计其叫声次数。

```
public class DuckSimulator {
    public static void main(String[] args) {
        DuckSimulator simulator = new DuckSimulator();
        simulator.simulate();
    }

    void simulate() {
        Quackable mallardDuck = new QuackCounter(new MallardDuck());
        Quackable redheadDuck = new QuackCounter(new RedheadDuck());
        Quackable duckCall = new QuackCounter(new DuckCall());
        Quackable rubberDuck = new QuackCounter(new RubberDuck());
        Quackable gooseDuck = new GooseAdapter(new Goose());

        System.out.println("\nDuck Simulator: With Decorator");

        simulate(mallardDuck);
        simulate(redheadDuck);
        simulate(duckCall);
        simulate(rubberDuck);
        simulate(gooseDuck);

        System.out.println("The ducks quacked " +
            QuackCounter.getQuacks() + " times");
    }

    void simulate(Quackable duck) {
        duck.quack();
    }
}
```

每次我们创建一个Quackable，就用一个新的装饰者包装它。

公园巡逻员告诉我们，他不想计入鹅的叫声，因此我们不去装饰鹅。

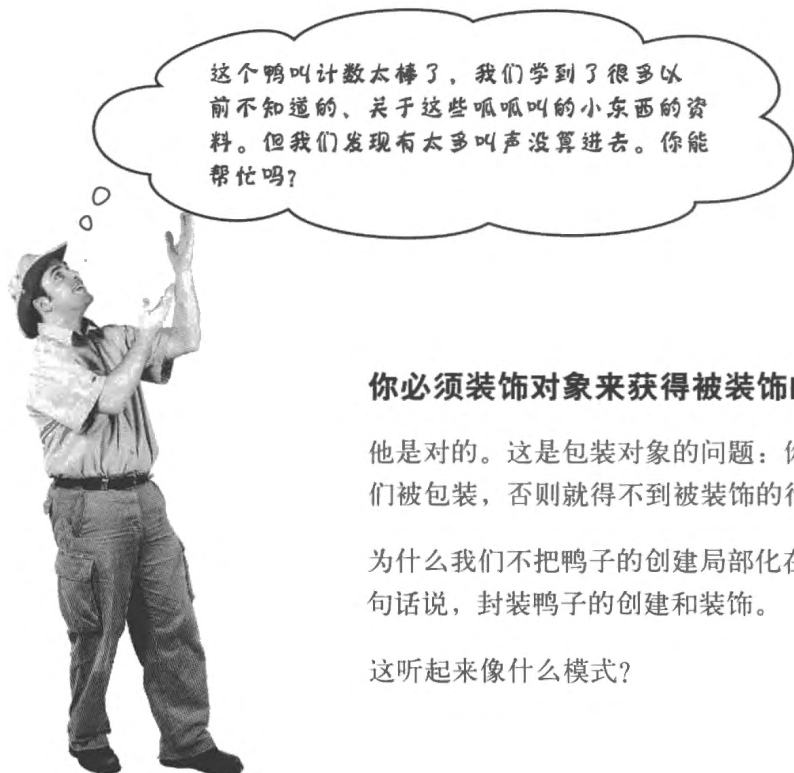
在这里，我们为呱呱叫学家收集呱呱叫行为。

这里没有任何变化，被装饰的对象还是Quackable。

输出在这里！

别忘了，鹅的叫声不计数。

```
File Edit Window Help DecoratedEggs
% java DuckSimulator
Duck Simulator: With Decorator
Quack
Quack
Kwak
Squeak
Honk
The ducks quacked 4 times
%
```



这个鸭叫计数太棒了，我们学到了很多以前不知道的、关于这些呱呱叫的小东西的资料。但我们发现有太多叫声没算进去。你能帮忙吗？

你必须装饰对象来获得被装饰的行为。

他是对的。这是包装对象的问题：你不得不确认它们被包装，否则就得不到被装饰的行为。

为什么我们不把鸭子的创建局部化在一个地方？换句话说，封装鸭子的创建和装饰。

这听起来像什么模式？

⑩ 我们需要工厂来生产鸭子！

好了！我们需要一些质量控制，以确保鸭子得到包装。我们要建造一个完整的工厂，专门生产这些鸭子。该工厂应该生产产品家族，包含不同类型的鸭子，因此我们要用抽象工厂模式。

让我们从AbstractDuckFactory类的定义开始：

```
public abstract class AbstractDuckFactory {  
  
    public abstract Quackable createMallardDuck();  
    public abstract Quackable createRedheadDuck();  
    public abstract Quackable createDuckCall();  
    public abstract Quackable createRubberDuck();  
}
```

我们定义一个抽象工厂，它的子类会实现不同的家族。

每个方法创建一种鸭子。

接下来，我们创建一个工厂，该工厂创建没有装饰者的鸭子，只是为了熟悉工厂的门道：

```
public class DuckFactory extends AbstractDuckFactory {
```

```
    public Quackable createMallardDuck() {
        return new MallardDuck();
    }
```

```
    public Quackable createRedheadDuck() {
        return new RedheadDuck();
    }
```

```
    public Quackable createDuckCall() {
        return new DuckCall();
    }
```

```
    public Quackable createRubberDuck() {
        return new RubberDuck();
    }
```

```
}
```

DuckFactory扩展
抽象工厂。

每个方法创建一种产品：一个特定
种类的Quackable。模拟器并不知道
实际的产品是什么，只知道得到了
一个Quackable。

现在，创建我们真正需要的工厂，CountingDuckFactory：

```
public class CountingDuckFactory extends AbstractDuckFactory {
```

```
    public Quackable createMallardDuck() {
        return new QuackCounter(new MallardDuck());
    }
```

```
    public Quackable createRedheadDuck() {
        return new QuackCounter(new RedheadDuck());
    }
```

```
    public Quackable createDuckCall() {
        return new QuackCounter(new DuckCall());
    }
```

```
    public Quackable createRubberDuck() {
        return new QuackCounter(new RubberDuck());
    }
```

```
}
```

CountingDuckFactory也
扩展抽象工厂。

每个方法用鸭叫计数装饰
器包装Quackable。模拟器
绝对不知道其中差别，只
知道取回一个Quackable。
但现在巡逻员可以确定，
所有的叫声都会被计数。

11 设置模拟器来使用工厂。

还记得抽象工厂怎么工作吗？我们创建一个多态方法，以一个工厂作为参数，并用它来创建对象。通过传入不同的工厂，就会得到不同的产品家族。

我们要修改simulate()方法，让它用传进来的工厂来创建鸭子。

```
public class DuckSimulator {
    public static void main(String[] args) {
        DuckSimulator simulator = new DuckSimulator();
        AbstractDuckFactory duckFactory = new CountingDuckFactory();

        simulator.simulate(duckFactory);
    }

    void simulate(AbstractDuckFactory duckFactory) {
        Quackable mallardDuck = duckFactory.createMallardDuck();
        Quackable redheadDuck = duckFactory.createRedheadDuck();
        Quackable duckCall = duckFactory.createDuckCall();
        Quackable rubberDuck = duckFactory.createRubberDuck();
        Quackable gooseDuck = new GooseAdapter(new Goose());

        System.out.println("\nDuck Simulator: With Abstract Factory");

        simulate(mallardDuck);
        simulate(redheadDuck);
        simulate(duckCall);
        simulate(rubberDuck);
        simulate(gooseDuck);

        System.out.println("The ducks quacked " +
            QuackCounter.getQuacks() +
            " times");
    }

    void simulate(Quackable duck) {
        duck.quack();
    }
}
```

首先，我们创建工厂，准备把它传入simulate()方法。

simulate()方法需要一个AbstractDuckFactory参数，用它创建鸭子，而不是直接实例化鸭子。

这里没变，一样的代码。

这里是使用工厂后的输出……

和上次一样，但是这次我们确定所有的鸭子都被装饰过，因为我们使用了 `CountingDuckFactory`。

```
File Edit Window Help EggFactory
% java DuckSimulator
Duck Simulator: With Abstract Factory
Quack
Quack
Kwak
Squeak
Honk
4 quacks were counted
%
```



削尖你的铅笔

我们仍然依赖具体类来直接实例化鹅。你能够为鹅写一个抽象工厂吗？怎样创建“名鸭实鹅”？

要分别管理这些不同的鸭子变得有些困难了，你能够帮我们想个办法，把这些鸭子作为一个整体来管理，甚至让我们能够管理几个想持续追踪的鸭子家族吗？



啊哈！他想管理一群鸭子。

巡视员又给咱们出了个好问题：为什么我们要个别管理鸭子？

这不太好管理呢！



```
Quackable mallardDuck = duckFactory.createMallardDuck();
Quackable redheadDuck = duckFactory.createRedheadDuck();
Quackable duckCall = duckFactory.createDuckCall();
Quackable rubberDuck = duckFactory.createRubberDuck();
Quackable gooseDuck = new GooseAdapter(new Goose());

simulate(mallardDuck);
simulate(redheadDuck);
simulate(duckCall);
simulate(rubberDuck);
simulate(gooseDuck);
```

我们需要一种方法来谈论鸭子集合，甚至子集合（为了满足巡视员管理鸭子家族的要求）。如果操作可以作用于整个集合的鸭子，那应该是很棒的事情。

什么模式可以帮到我们？

12

我们来创建一群鸭子（好吧，实际上是一群Quackable）。

还记得，组合模式允许我们将一个对象集合像单个对象一样对待吗？对一群Quackable来说，组合模式最合适不过了！

让我们逐步看如何运作：

```
public class Flock implements Quackable {
    List<Quackable> quackers = new ArrayList<Quackable>();

    public void add(Quackable quacker) {
        quackers.add(quacker);
    }

    public void quack() {
        Iterator<Quackable> iterator = quackers.iterator();
        while (iterator.hasNext()) {
            Quackable quacker = iterator.next();
            quacker.quack();
        }
    }
}
```

别忘了，组合需要和叶元素一样实现相同的接口。我们的叶元素是Quackable。

在每一个Flock内，我们使用ArrayList持有属于这个Flock的Quackable对象。

add()方法添加一个Quackable对象到Flock。

现在轮到quack()方法，毕竟Flock也是Quackable。Flock中的quack()方法需要对整个群产生作用。在这里，我们遍历ArrayList，调用每一个元素上的quack()。



Code up Close

你注意到了吗，我们其实还偷偷用了另一个设计模式，只是没有告诉你？

```
public void quack() {
    Iterator<Quackable> iterator = quackers.iterator();
    while (iterator.hasNext()) {
        Quackable quacker = iterator.next();
        quacker.quack();
    }
}
```

在这里！迭代器模式在工作！

13 现在我们需要修改模拟器。

我们的组合准备好了，我们只需要一些代码，把鸭子归入组合结构。

```
public class DuckSimulator {
    // 这里是主要方法
```

```
void simulate(AbstractDuckFactory duckFactory) {
    Quackable redheadDuck = duckFactory.createRedheadDuck();
    Quackable duckCall = duckFactory.createDuckCall();
    Quackable rubberDuck = duckFactory.createRubberDuck();
    Quackable gooseDuck = new GooseAdapter(new Goose());

    System.out.println("\nDuck Simulator: With Composite - Flocks");
```

和之前一
样，创建所有
Quackable。

```
Flock flockOfDucks = new Flock();
```

```
flockOfDucks.add(redheadDuck);
flockOfDucks.add(duckCall);
flockOfDucks.add(rubberDuck);
flockOfDucks.add(gooseDuck);
```

首先，我们创建一个Flock，
并给它加载许多Quackable。

```
Flock flockOfMallards = new Flock();
```

然后创建一个新的
绿头鸭群。

```
Quackable mallardOne = duckFactory.createMallardDuck();
Quackable mallardTwo = duckFactory.createMallardDuck();
Quackable mallardThree = duckFactory.createMallardDuck();
Quackable mallardFour = duckFactory.createMallardDuck();
```

在这里，我们创
建一个绿头鸭的
小家族。

```
flockOfMallards.add(mallardOne);
flockOfMallards.add(mallardTwo);
flockOfMallards.add(mallardThree);
flockOfMallards.add(mallardFour);
```

把它们加入绿头鸭群。

```
flockOfDucks.add(flockOfMallards);
```

把绿头鸭群加入主群。

```
System.out.println("\nDuck Simulator: Whole Flock Simulation");
simulate(flockOfDucks);
```

测试整个群。

```
System.out.println("\nDuck Simulator: Mallard Flock Simulation");
simulate(flockOfMallards);
```

然后只测试绿头鸭群。

```
System.out.println("\nThe ducks quacked " +
    QuackCounter.getQuacks() +
    " times");
```

最后，把数据给呱呱叫
学家。

```
}
```

```
void simulate(Quackable duck) {
    duck.quack();
}
```

这里不需要修改，Flock是一个Quackable!

执行结果……

```

File Edit Window Help FlockADuck
% java DuckSimulator

Duck Simulator: With Composite - Flocks
Duck Simulator: Whole Flock Simulation
Quack
Kwak
Squeak
Honk
Quack
Quack
Quack
Quack

Duck Simulator: Mallard Flock Simulation
Quack
Quack
Quack
Quack

The ducks quacked 11 times

```

这里是第一个群。

现在是绿头鸭。

数据看起来是对的
(别忘了, 鹅是不计数的)。



安全与透明

你可能还记得, 在组合模式章节中, 组合(菜单)以及叶子(菜单项)有一组完全相同的方法, 包括了add()方法。因为它们有一组相同的方法, 我们可以在菜单项上调用不起作用的方法(像试图通过调用add()给菜单项添加一些东西)。这样的好处是, 叶子和组合之间的区分是透明的, 客户不必知道它处理的是组合还是叶子, 它只是用同一个方法来调用两者。

在这里, 我们决定把组合维护孩子的方法和叶结点分开: 即, 只有Flock有add()方法, 我们知道, 试图给一个Duck添加某些东西是无意义的, 在这个实现中, 就没法这样做。你只能对Flock做add()。因此, 这个设计更安全(你不能调用组件上无意义的方法), 但透明性比较差。现在, 为了添加Quackable, 客户不得不知道Quackable是Flock。



组合运作得很顺畅！谢谢！现在我们有一个相反的要求：我们还需要跟踪个别的鸭子。你有办法让我持续跟踪个别鸭子的实时嘎嘎叫吗？

你会说“观察者”吗？

听起来，呱呱叫学家想要观察个别鸭子的行为，这让我们想起有一个模式可以观察对象的行为：观察者模式。

14 首先我们的主题（Subject）需要一个接口。

记住，主题是被观察的对象。为了好记，我们称为Observable如何？Observable需要有注册和通知观察者的方法。也可以有移除观察者的方法，但是在这里为了让实现保持简单，省略了。

```
public interface QuackObservable {  
    public void registerObserver(Observer observer);  
    public void notifyObservers();  
}
```

QuackObservable是一个接口，想被观察的Quackable，应该实现它。

它也有一个通知观察者的方法。

它有一个注册观察者的方法，任何实现Observer接口的对象都可以监听呱呱叫。我们稍后会定义观察者接口。

现在 我们需要确认所有的Quackable都实现此接口。

```
public interface Quackable extends QuackObservable {  
    public void quack();  
}
```

所以，我们用QuackObserver来扩展Quackable接口。

- 15 现在，我们需要确定所有实现Quackable的具体类，都能够扮演QuackObservable。

为了达到这个目的，可以通过在每一个类中实现注册和通知（像第2章所做的）。但是这次我们的做法稍微不同：我们要在另一个类封装注册和通知的代码，这个类称为Observable，然后把它和QuackObservable组合。这样，我们只写一次真实的代码，QuackObservable只需要把调用委托给辅助类Observable。

我们先从Observable辅助类开始……

Observable实现了所有必要的功能。我们只需把它插进一个类，就可以让该类将工作委托给Observable。

Observable必须实现QuackObservable，因为它们会被委托相同的方法调用。

```
public class Observable implements QuackObservable {
    List<Observer> observers = new ArrayList<Observer>();
    QuackObservable duck;

    public Observable(QuackObservable duck) {
        this.duck = duck;
    }

    public void registerObserver(Observer observer) {
        observers.add(observer);
    }

    public void notifyObservers() {
        Iterator iterator = observers.iterator();
        while (iterator.hasNext()) {
            Observer observer = iterator.next();
            observer.update(duck);
        }
    }
}
```

在构造器中，我们传入QuackObservable，用这个对象管理可观察行为。看看下面的notifyObservers()方法，你会看到，当通知发生时，Observable把这个对象传过去，好让观察者知道是哪个对象在呱呱叫。

这是注册观察者的代码。

这是通知的代码。

接下来，我们来看看Quackable类是如何使用这个辅助类的……



16 整合Observable辅助类和Quackable类。

这应该不算太糟。我们只需要确定Quackable类是和Observable组合在一起，并知道如何委托工作，然后，它们就准备好成为Observable了。下面是MallardDuck的实现，其他的鸭子也相同。

```
public class MallardDuck implements Quackable {
    Observable observable;

    public MallardDuck() {
        observable = new Observable(this);
    }

    public void quack() {
        System.out.println("Quack");
        notifyObservers();
    }

    public void registerObserver(Observer observer) {
        observable.registerObserver(observer);
    }

    public void notifyObservers() {
        observable.notifyObservers();
    }
}
```

每个Quackable都有一个Observable实例变量。

在构造器中，我们创建一个Observable，并传入一个MallardDuck对象的引用。

当我们呱呱叫时，需要让观察者知道。

这是我们的两个QuackObservable方法。注意，我们只是委托给辅助类。



削尖你的铅笔

我们还没有改变一个Quackable的实现，即QuackCounter装饰者，它也需要成为Observable。你为什么不写出它的代码呢？

17 几乎大功告成了！我们只需要把模式的Observer端完成。

我们已经实现了Observable所需要的一切，现在需要一些观察者（Observer），我们从Observer接口开始：

Observer接口只有一个方法，update()，需要传入正在呱呱叫的QuackObservable。

```
public interface Observer {
    public void update(QuackObservable duck);
}
```

现在我们需要一个观察者：呱呱叫学家跑哪里去了？

我们需要实现Observer接口，否则就无法以QuackObservable注册。

```
public class Quackologist implements Observer {

    public void update(QuackObservable duck) {
        System.out.println("Quackologist: " + duck + " just quacked.");
    }
}
```

Quackologist很简单，只有一个方法update()，它打印出刚才呱呱叫的Quackable。



削尖你的铅笔

要是呱呱叫学家想观察整个群怎么办？这意味着什么？像这样思考：如果我们观察一个组合，那么就是观察组合内的每个东西。因此，当你注册要观察某个群，群组合确定让你能观察所有的孩子（抱歉，所有小呱呱叫者），这可能还包括其他群。

在进入后面的内容前，请编写Flock观察者的代码。

⑱ 我们准备观察了。让我们更新模拟器并尝试一下：

```
public class DuckSimulator {
    public static void main(String[] args) {
        DuckSimulator simulator = new DuckSimulator();
        AbstractDuckFactory duckFactory = new CountingDuckFactory();

        simulator.simulate(duckFactory);
    }

    void simulate(AbstractDuckFactory duckFactory) {

        // 在这里创建鸭子工厂和鸭子

        // 在这里创建群

        System.out.println("\nDuck Simulator: With Observer");

        Quackologist quackologist = new Quackologist();
        flockOfDucks.registerObserver(quackologist);

        simulate(flockOfDucks);

        System.out.println("\nThe ducks quacked " +
            QuackCounter.getQuacks() +
            " times");
    }

    void simulate(Quackable duck) {
        duck.quack();
    }
}
```

我们这里所需要做的是
创建一个Quackologist，
把它设置为群的观察者。

这次我们模拟
整个群。

让我们试一下，看看这一切
是如何工作的！

这是一个大场面的终局。五个，不，有六个模式一起来创建这个令人惊叹的鸭子模拟器。在没有更多的麻烦下，我们呈现鸭子模拟器！

```
File Edit Window Help DucksAreEverywhere

% java DuckSimulator

Duck Simulator: With Observer
Quack
Quackologist: Redhead Duck just quacked.
Kwak
Quackologist: Duck Call just quacked.
Squeak
Quackologist: Rubber Duck just quacked.
Honk
Quackologist: Goose pretending to be a Duck just quacked.
Quack
Quackologist: Mallard Duck just quacked.
Quack
Quackologist: Mallard Duck just quacked.
Quack
Quackologist: Mallard Duck just quacked.
Quack
Quackologist: Mallard Duck just quacked.
Quackologist: Mallard Duck just quacked.
The Ducks quacked 7 times.
```

在每一次呱呱叫后，不管是哪一种呱呱叫，观察者都会收到通知。

呱呱叫学家依然会得到计数。

没有蠢问题

问：所以这就是复合模式？

答：不，这只是一群模式的合作。复合模式是指一些模式被结合起来解决一般性问题。我们很快就会看到 Model-View-Controller（模型-视图-控制器）复合模式，它是一些模式的集合，在许多设计解决方案中一再被使用。

问：所以真正的设计模式之美在于遇到问题时就开始对它应用模式，直到得到解决方案。对吗？

答：错。我们在鸭子练习中这么做，是为了展示模式可以合作。在实际工作中，你绝不会想要像刚才那样做设计。事实上，鸭子模拟器的各部分都有许多解决方案，只是这些模式中有一些是“杀鸡焉用宰牛刀”。有时候，只要使用良好的OO设计原则就可以解决问题，这样其实就够了。

下一章，我们将讨论更多这方面的问题，你应该只在合适的时间和地点应用模式，绝对不能为了使用模式而使用模式。这样想来，鸭子模拟器的设计看起来就显得做作，但是，这个例子有趣，并且让我们很好地了解若干模式如何匹配到一个解决方案。

我们做了什么？

我们从一大堆Quackable开始……

有一只鹅出现，希望自己也像一个Quackable。因此我们用适配器模式把鹅适配成Quackable，就可以调用适配器的quack()方法，让鹅咯咯叫。

然后，呱呱叫学家决定要计算呱呱叫的次数。因此我们用装饰者模式，添加了一个QuackCounter装饰者，用来跟踪quack()被调用的次数，并将调用委托给它所包装的Quackable。

但是呱呱叫学家担心忘记加上QuackCounter装饰者。所以我们用抽象工厂模式创建鸭子。从此，无论何时需要鸭子，就找工厂要，工厂会交回装饰过的鸭子（别忘了，如果要没被装饰的鸭子，也可以用另一个鸭子工厂）。

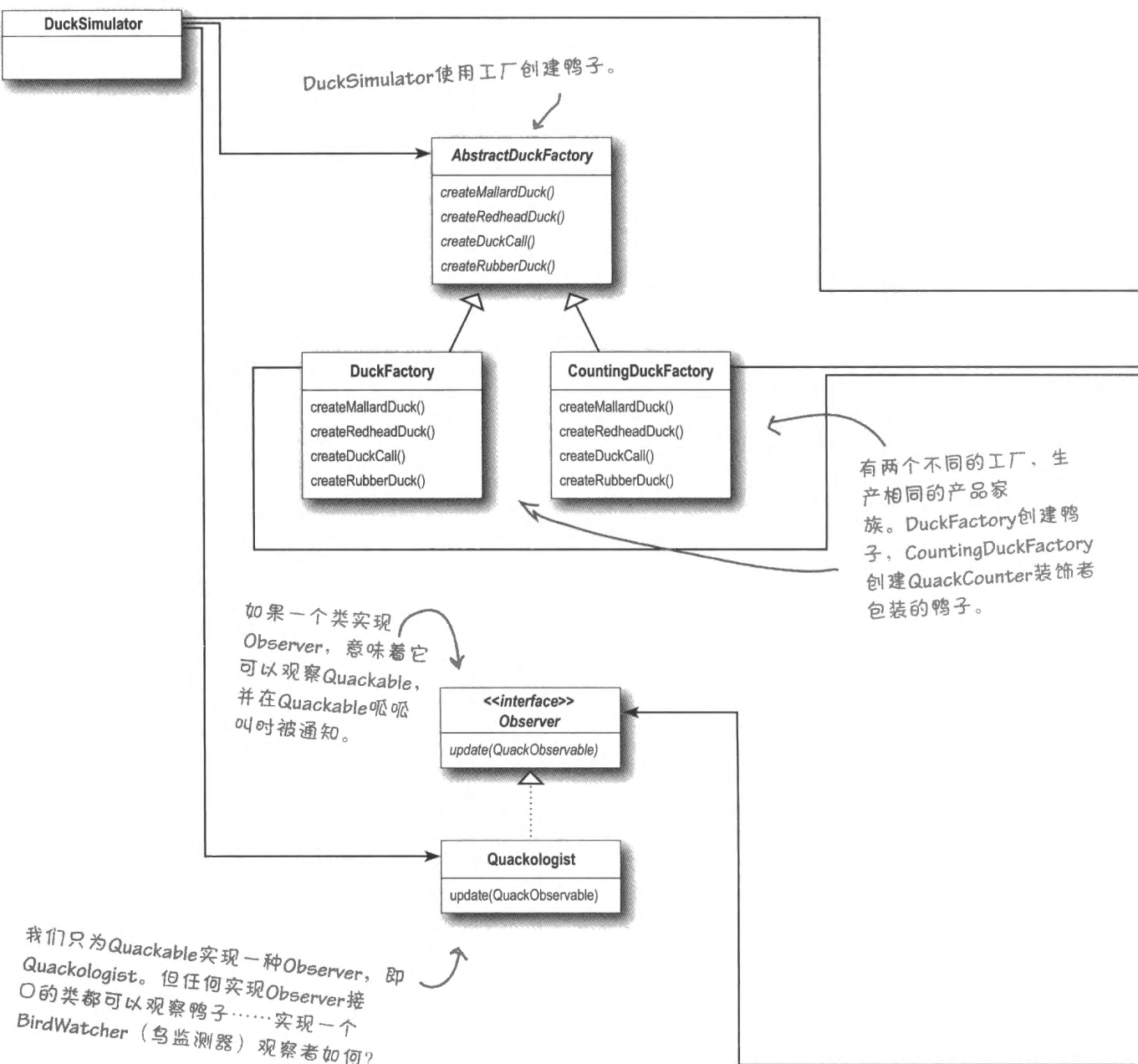
跟踪所有这些鸭子、鹅和Quackable，我们面临麻烦的管理问题。因此我们用组合模式来把Quackable分组成群。这个模式也允许嘎嘎叫学家创建子群，以便管理鸭子家族。通过使用java.util.ArrayList的迭代器，我们在实现中使用了迭代器模式。

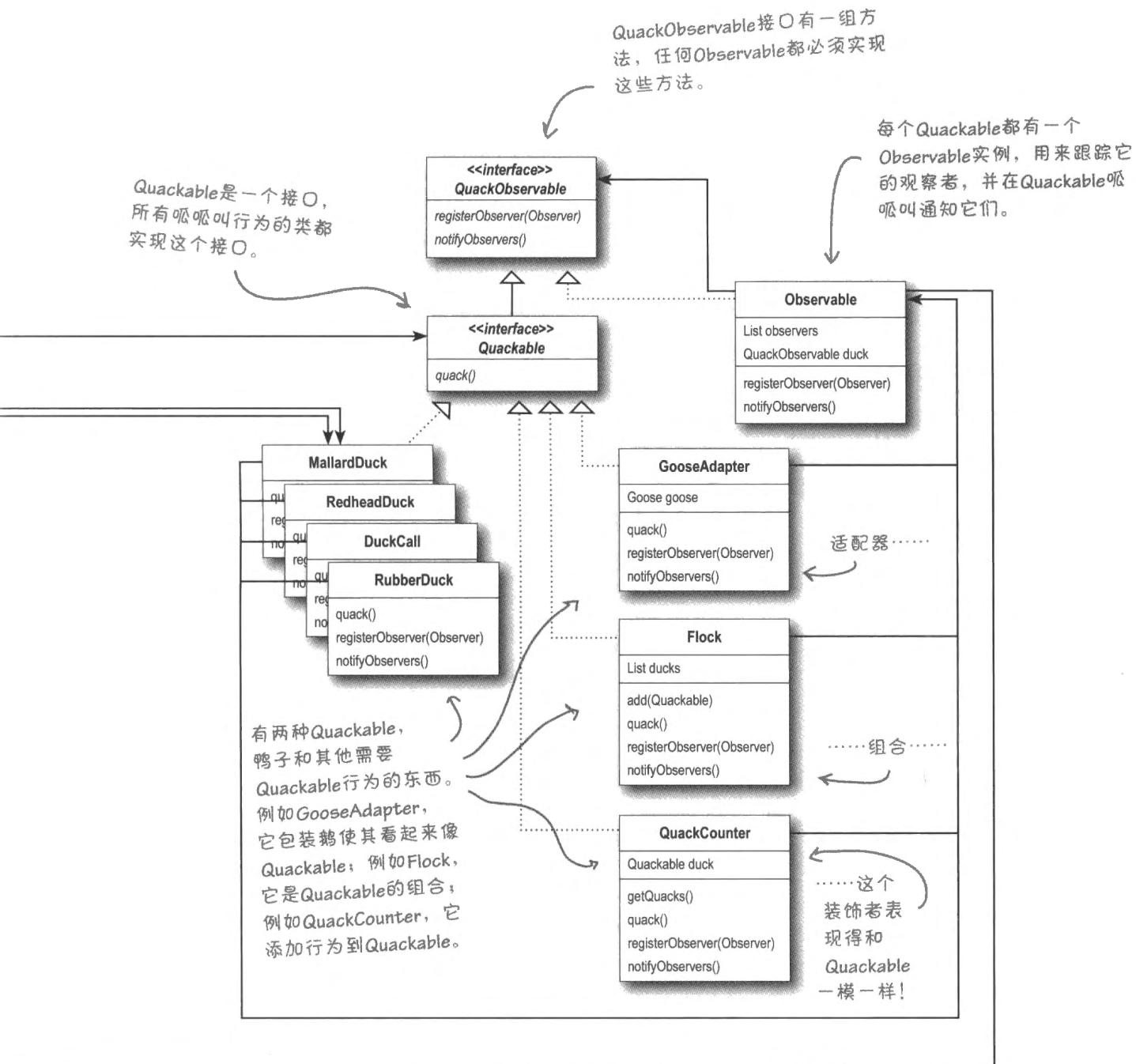
每次Quackable呱呱叫时，呱呱叫学家希望能被通知。因此我们使用观察者模式，让呱呱叫学家注册成为Quackable观察者。现在，每次Quackable呱呱叫时，呱呱叫学家就会被通知。在这个实现中，我们再次用到迭代器。呱呱叫学家甚至可以对组合使用观察者模式。



~~鸟~~鸭瞰视图: 类图

在一个小小的鸭子模拟器中，我们打包了许多模式。大局是这样的：





复合模式之王

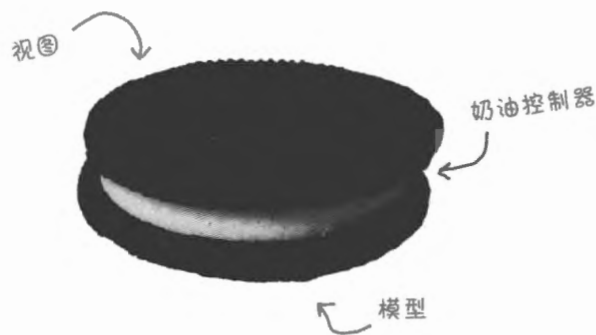
如果猫王是复合模式，他的名字会叫做模型-视图-控制器，他会唱这么一首歌……

模型 - 视图 - 控制器

词曲：James Dempsey

MVC是一种范型

它构造代码成为功能段，免得你脑子炸掉
为达到复用，你必须让边界干净
一边模型，另一边视图，控制器在中间



模型 视图，和奥利奥一样有三层
模型 视图 控制器
模型 视图， 模型 视图， 模型 视图 控制器

模型对象正是你的应用系统存在的理由
定制的对象，包含了数据、逻辑，之类的
在应用的问题域，你创建定制类
你可以选择复用所有视图
但模型对象不用改变

你可以建模一辆汽车的节气门或者排气管
建模一个两岁小孩
建模一瓶上等的霞多丽葡萄酒

建模人们的窃窃私语
建模一些水煮蛋
建模Hexley的步履蹒跚

模型 视图

你可以建模GQ时尚杂志中的模特。

模型 视图 控制器

Java也是!

视图对象通常是控件，用来显示和编辑
Cocoa^{译注1}是这样，设计良好，赢得好评
把任何老的Unicode字符串交给NSTextView^{译注2}
用户可以和它交互，它几乎可以持有任何东西
但视图不知道模型
字符串可以是一个电话号码，或者亚里士多德的文学作品
保持松耦合
达到最大级别的复用

模型 视图，一切都是水波荡漾的蓝色^{译注3}
模型 视图 控制器

你也许怀疑现在要不要做？
你也许怀疑怎么做到？
数据在模型和视图之间流动
控制器居间协调
每层之间的状态改变
为了同步两者的数据
它推拉每个改变值

译注1：Cocoa是MacOS X的面向对象API。

译注2：这是Cocoa中的一个类。

译注3：这是MacOS X的用户界面Aqua预定的颜色。

模型视图，Smalltalk人的疯狂道具

模型视图控制器

模型视图，读作“噢噢”，不是“哦哦”

模型视图控制器

故事还没结束

前方还有道路

编写控制器代码

似乎没给人带来太多荣耀

模型的使命至关重要

视图很华丽

我或许很懒，但有时却是疯了

我写了多少代码，只是为了粘合

这不是那么惨痛

而且代码没什么魔法

只是搬动值

我无意变得狂躁

但它一再重复

做控制器做的所有事情

我真希望能得到一毛钱

每次将字符串值发给TextField

模型视图

我们如何对黏合置之不理

模型视图控制器

控制器非常了解模型和视图

它们经常用硬编码，这对复用不利

但现在你可以连接所选每个模型的键到任何视图属性

一旦开始绑定

你会发现源码树上的代码变少了

耶，这一切自动又免费，我兴高采烈

我想它负责重复所有你不会需要的代码

当你把它挂接到IB时

 用Swing。

模型视图，甚至也处理多重选择

模型视图控制器

模型视图，敢打赌我比你先交付

模型视图控制器



动动耳

别光读歌词！毕竟这是一本HeadFirst书……到这个URL：

<https://www.youtube.com/watch?v=YYvOGPMLVDo>

靠着椅背，听歌。

很有趣的歌，但这真的能教我模型-视图-控制器吗？我之前尝试过学习MVC，学得很头痛。



设计模式是理解MVC的钥匙。

我们只是用这首歌来刺激一下你的胃口。你读完本章之后，再回头去听这首歌，会觉得更有趣。

听起来你在MVC上有过不好的遭遇？我们大多数人都有。你可能听其他开发人员说过，MVC改变了他们的生活，甚至可能创造世界和平。它是一个威力强大的复合模式，确实，我们不能抱怨它没有带来世界和平，不过一旦你懂得了MVC，可以节省你编码的时间。

但首先得学会它，是吧？嗯，这次的学习会大大不同，因为你现在已经懂得模式了！

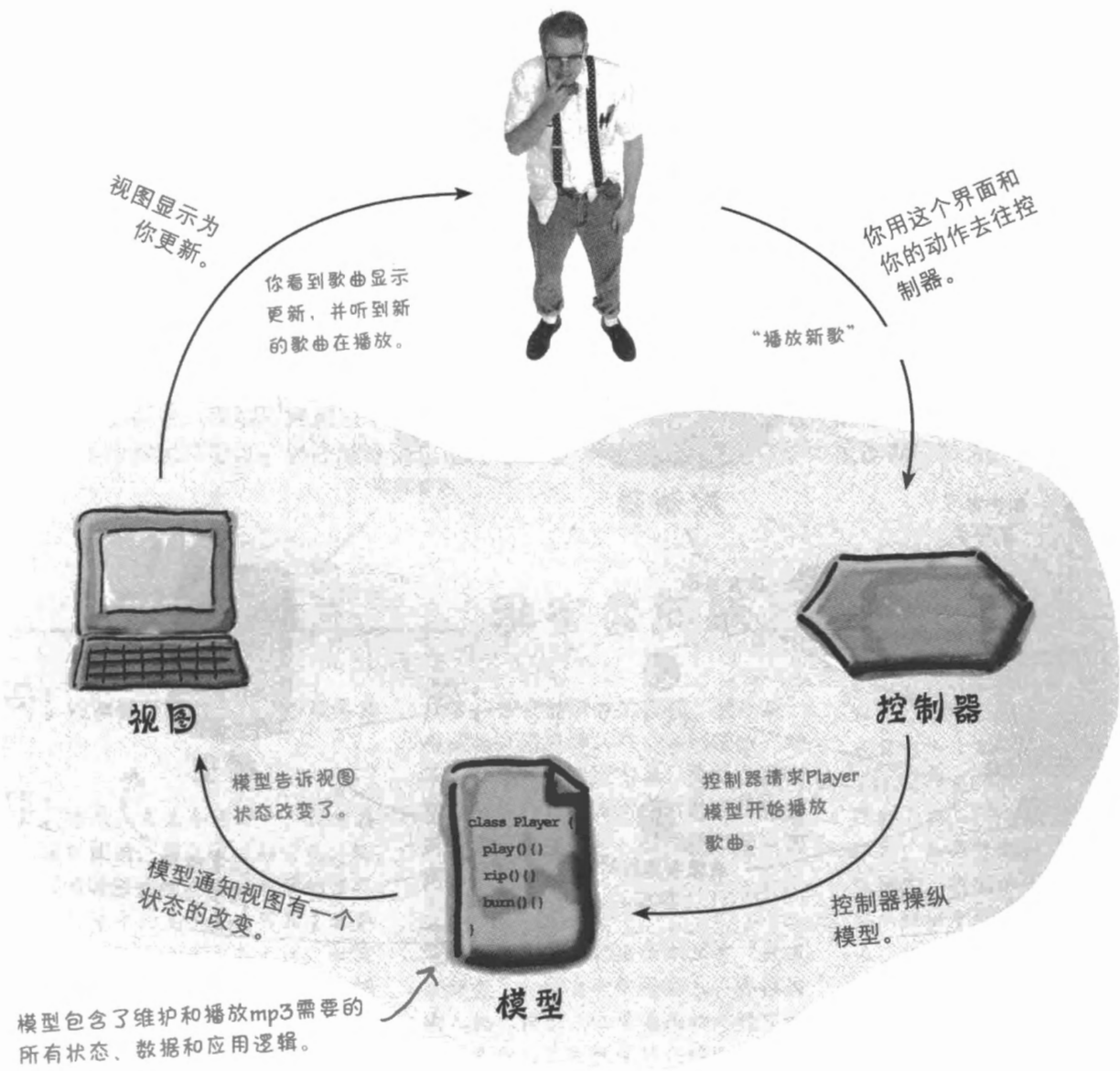
没错，设计模式是MVC的钥匙。想要自顶而下学习MVC是困难的，成功的开发人员不多。学习MVC的诀窍是，它是由若干模式结合起来的。当你通过查看这些模式来学习MVC时，刹那间就开始有感觉了。

我们开始。这一次你要拿下MVC！

认识模型-视图-控制器

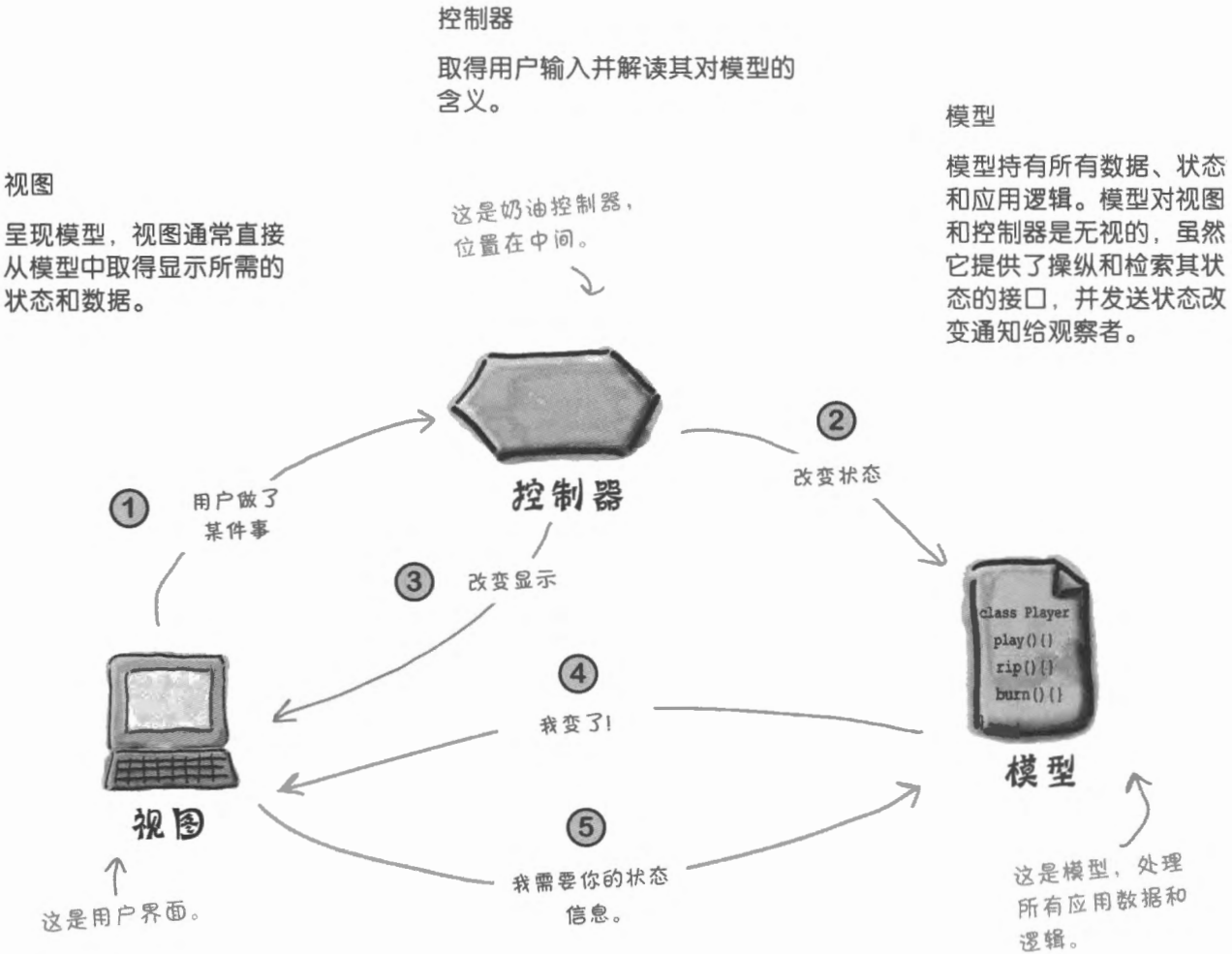
想象你正在使用你最喜欢的音乐播放器，例如iTunes。你可以用它的界面来添加新的歌曲、管理播放列表、给歌曲改名。播放器维护一个小型数据库，里面有所有的歌曲以及相关的名称和数据。播放器也可以播放歌曲，此时，用户界面会显示不断更新的歌曲标题、运行时间等。

好吧，它底下就是模型-视图-控制器……



靠近一点……

音乐播放器的描述，给了我们一个MVC的高级别视图，但是这真的不能帮助你理解复合模式内的运作细节，以及如何创建自己的复合模式，以及为什么复合模式如此之好。让我们从模型、视图、控制器三者的关系开始，然后再次从设计模式的视角来看。



- ① **你是用户，你和视图交互。**
视图是你看模型的窗口，当你对视图做一些事时（例如点击播放按钮），视图就告诉控制器你做了什么，控制器的工作是负责处理。
- ② **控制器要求模型改变状态。**
控制器接受并解读你的动作，如果你点击某个按钮，控制器的任务就是理解这个动作的含义，以及如何基于此动作操纵模型。
- ③ **控制器也可能要求视图做改变。**
作为控制器从视图接收到动作的结果，控制器可能需要告诉视图改变。例如，控制器可以使得界面上的某些按钮或菜单项有效或无效。
- ④ **当模型状态改变时，模型通知视图。**
不管是基于一些你所做的动作（例如点击按钮），还是其他内部改变（例如播放列表的下一首歌开始），当模型中某些东西改变时，模型通知视图它的状态已改变。
- ⑤ **视图向模型询问状态。**
视图直接从模型取得要显示的状态。例如，当模型通知视图新歌开始播放，视图向模型询问歌名并显示。当控制器请求视图做某些改变时，视图也可能向模型询问状态。

没有蠢问题

问：控制器可以变成模型的观察者吗？

答：当然。在某些设计中，控制器会向模型注册，然后被通知变化。这是模型直接影响用户界面控件的情况。例如，模型中的某些状态可能支配界面的某些项目变成有效或无效。如果这样，要求视图更新相应的显示，确实是控制器的工作。

问：控制器所做的事情，就是从视图得到用户输入并发送到模型，对吗？如果只是做这些事，为什么要有控制器？为什么不把代码放在视图自身？大多数情况下，控制器不是只调用模型的方法吗？

答：控制器做的事情不只有“发送给模型”，它负责解读输入，并根据输入操纵模型。但你真正的问题可能是“为什么不在视图的代码中做这个？”

当然可以；但是，你不会想这么做，有两个原因：其一，这会让视图的代码更复杂，因为现在视图有两个责任：管理用户界面和处理如何控制模型的逻辑。其二，造成模型和视图之间紧耦合。如果你要把这个视图复用于其他模型，别指望了。控制器把控制逻辑从视图中分离，把视图从模型解耦。通过保持视图和控制器松耦合，你建造了更有弹性和扩展性的设计，更容易容纳以后的改变。

把MVC理解为一套模式

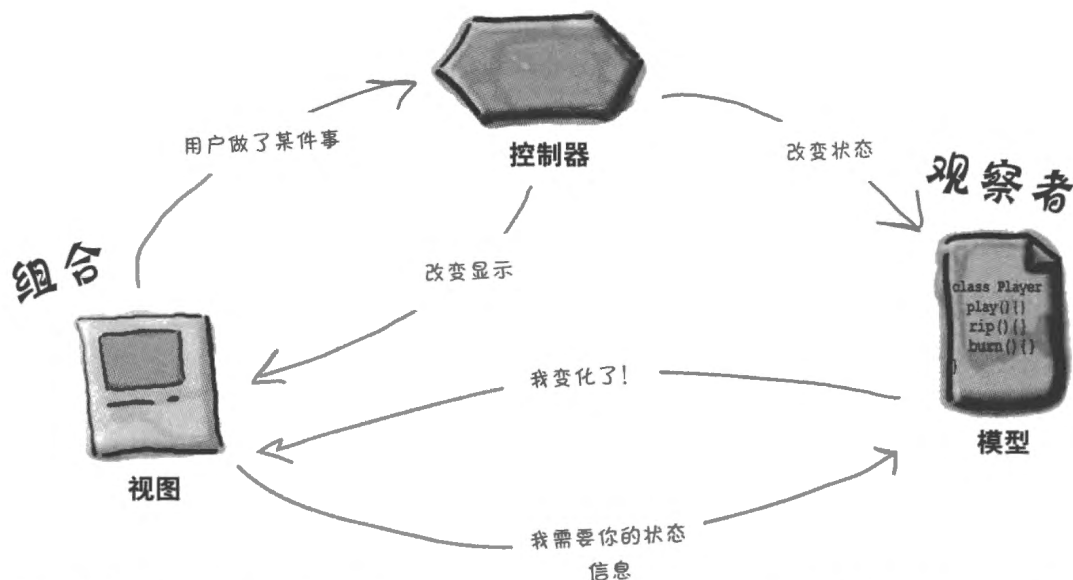
我们已经说过，学会MVC最好的方法，就是看看它是什么东西，在同一个设计中合作的一套模式。

让我们从模型开始，模型使用观察者来保持视图和控制器可以随最新的状态变化而更新。另外，视图和控制器实现了策略模式，控制器是视图的策略，如果你想要不同的行为，换一个控制器很容易。视图本身也内部使用一个模式来管理窗口、按钮，以及其他显示组件：组合模式。

我们来更详细地看一下：

策略

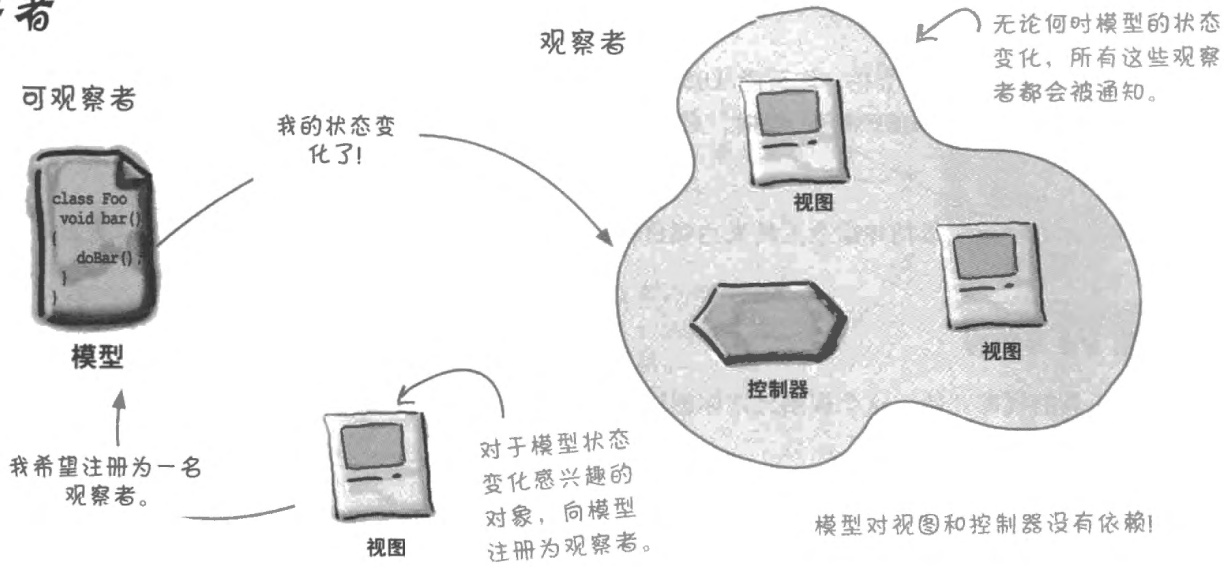
视图和控制器实现经典的策略模式，视图是一个对象，它用策略构造。控制器提供策略。视图只关心应用的可视方面，并把任何关于界面行为的决定委托给控制器。使用策略模式也保持视图从模型解耦，因为控制器负责和模型交互来执行用户请求。视图不知道这是怎么做到的。



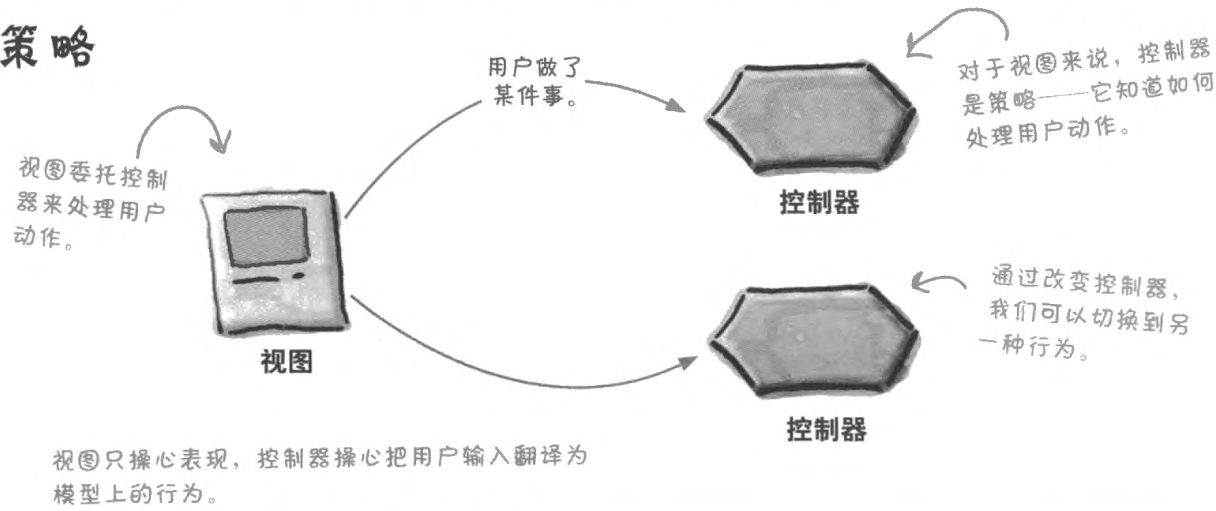
显示包括一个嵌套的窗口、面板、按钮、文本标签等等的集合。每个显示组件要么是组合（像窗口），要么是叶子（像按钮）。当控制器告诉视图更新时，只需告诉顶层的视图组件即可，组合会处理其他。

模型实现了观察者模式，当状态变化发生时，保持感兴趣的对象更新。使用观察者模式，使得模型完全独立于视图和控制器。它允许我们为同一个模型使用不同的视图，甚至同时使用多个视图。

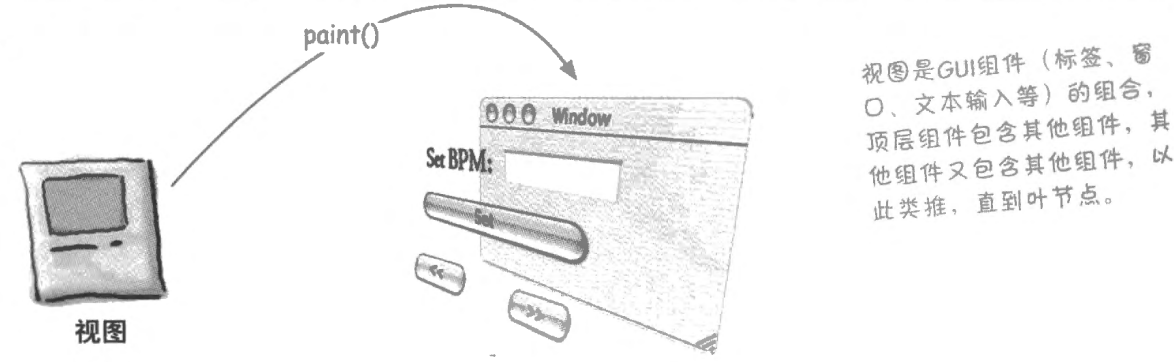
观察者



策略

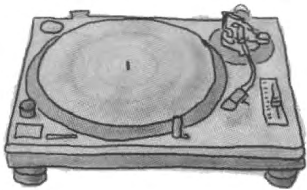


组合



使用MVC控制节拍

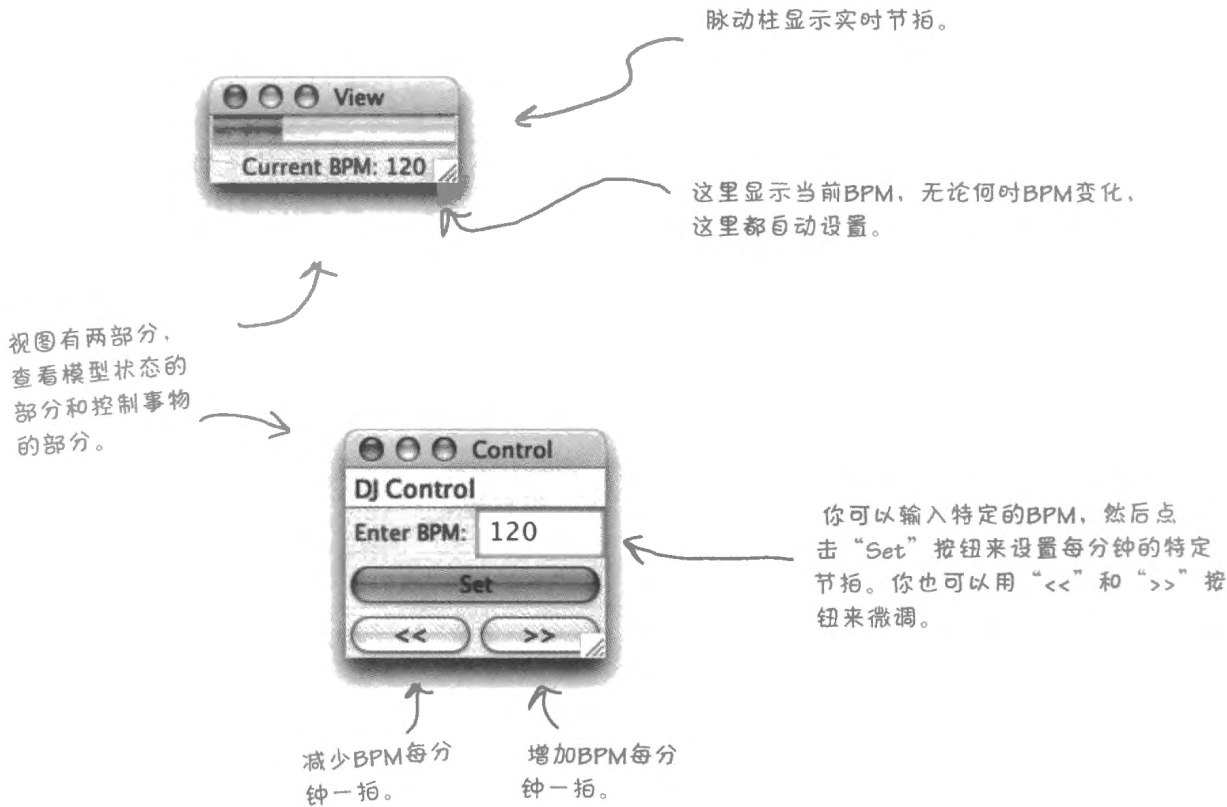
现在让你来当DJ。DJ的关键是节拍，你可能从95BPM（每分钟95拍）缓拍开始，然后把人群带到疯狂的140BPM迷幻舞曲，最后以柔和的80BPM混合音乐结束。



你打算怎么做？你必须控制节拍并建造工具来达到目标。

认识Java DJ View

让我们从这个工具的视图开始。这个视图允许你创建鼓声节拍，并调整其BPM……



这里有更多控制DJ View的方式……



你可以通过选择“DJ Control”菜单中的Start项开始节拍。

使用“Stop”按钮停止生成节拍。

注意，开始节拍之前，Stop是无效的。

注意，节拍开始后，Start是无效的。

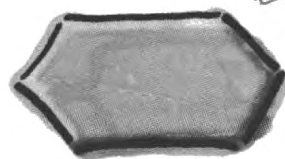


所有用户动作被发送到控制器。

控制器在中间……

控制器位于视图和模型之间，它将你的输入（像从“DJ Control”菜单中选择“Start”），转变成模型上的动作，启动节拍的产生。

控制器从用户取得输入，想办法翻译成对模型请求。



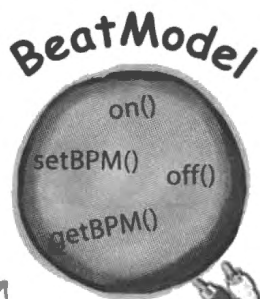
控制器

别忘了在下面的模型……

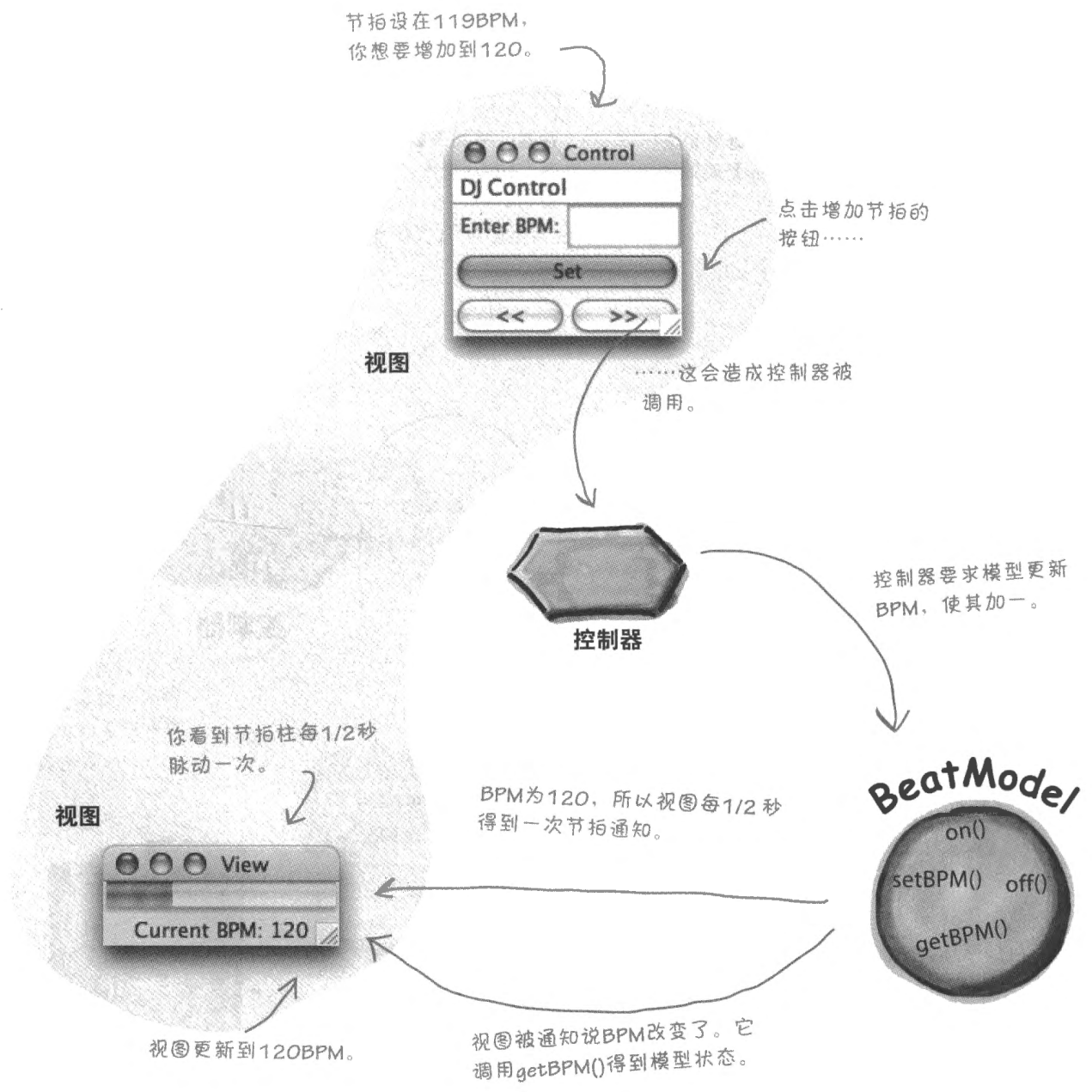
你看不到模型，但是听得到。模型在每样东西的最底层，管理节拍、驱动喇叭。

BeatModel是应用的核心，它实现了节拍开始和停止，设置BPM以及生成声音的逻辑。

该模型也允许我们通过getBPM()方法，取得它的当前状态。



把碎片拼起来



建造碎片

OK，现在你已经知道模型负责维护所有的数据、状态和应用逻辑。那BeatModel又是干什么的呢？它的主要工作是管理节拍，因此它有维护当前BPM的状态以及播放音频片段来创建我们所听到的节拍的代码。它也暴露一个接口，让控制器操纵节拍，让视图和控制器获得模型的状态。还有，别忘了模型使用观察者模式，因此我们也需要一些方法，让对象注册为观察者，以及送出通知。

在看实现之前，我们来了解一下

BeatModelInterface接口:

```
public interface BeatModelInterface {
```

BeatModel初始化后，这个方法被调用。

这些是控制器的方法，用来基于用户交互指引模型。

```
void initialize();
```

```
void on() ;
```

这些方法打开或关闭节拍产生器。

```
void off();
```

这个方法设置BPM。调用此方法后，节拍频率马上改变。

```
void setBPM(int bpm);
```

这些方法允许视图和控制器取得状态，并变成观察者。

```
int getBPM();
```

getBPM()方法返回当前BPM，如果节拍器是关闭的，返回0。

```
void registerObserver(BeatObserver o);
```

```
void removeObserver(BeatObserver o);
```

```
void registerObserver(BPMObserver o);
```

```
void removeObserver(BPMObserver o);
```

这看起来应该很熟悉。这些方法允许对象注册为状态改变的观察者。

我们把这分成两种观察者：一种观察者希望每个节拍都被通知，另一种观察者只希望BPM改变时被通知。

现在来看看具体的BeatModel类

我们实现了BeatModelInterface和Runnable。

```
public class BeatModel implements BeatModelInterface, Runnable {
    List<BeatObserver> beatObservers = new ArrayList<BeatObserver>();
    List<BPMObserver> bpmObservers = new ArrayList<BPMObserver>();
    int bpm = 90;
    Thread thread;
    boolean stop = false;
    Clip clip;

    public void initialize() {
        try {
            File resource = new File("clap.wav");
            clip = (Clip) AudioSystem.getLine(new Line.Info(Clip.class));
            clip.open(AudioSystem.getAudioInputStream(resource));
        }
        catch(Exception ex) { /* ... */ }
    }

    public void on() {
        bpm = 90;
        notifyBPMObservers();
        thread = new Thread(this);
        stop = false;
        thread.start();
    }

    public void off() {
        stopBeat();
        stop = true;
    }

    public void run() {
        while (!stop) {
            playBeat();
            notifyBeatObservers();
            try {
                Thread.sleep(60000/getBPM());
            } catch (Exception e) {}
        }
    }

    public void setBPM(int bpm) {
        this.bpm = bpm;
        notifyBPMObservers();
    }

    public int getBPM() {
        return bpm;
    }

    // 注册和通知观察者的代码
    // 处理节拍的音频代码
}
```

这些ArrayList持有两种观察者（节拍和BPM观察者）

BPM变量持有节拍的频率，缺省为90BPM。

这个方法设置节拍音轨。

on()方法设置BPM为默认值，并开始播放节拍的线程。

off()通过设置BPM为0，关闭并停止播放节拍的线程。

run()方法运行节拍线程，播放由BPM决定的节拍，并通知观察者节拍已播放。当我们从菜单选择Stop时，循环结束。

setBPM()方法是控制器操纵节拍的方式，它设置bpm变量，并通知所有BPM观察者BPM已经改变。

getBPM()方法仅返回当前每分钟的节拍。



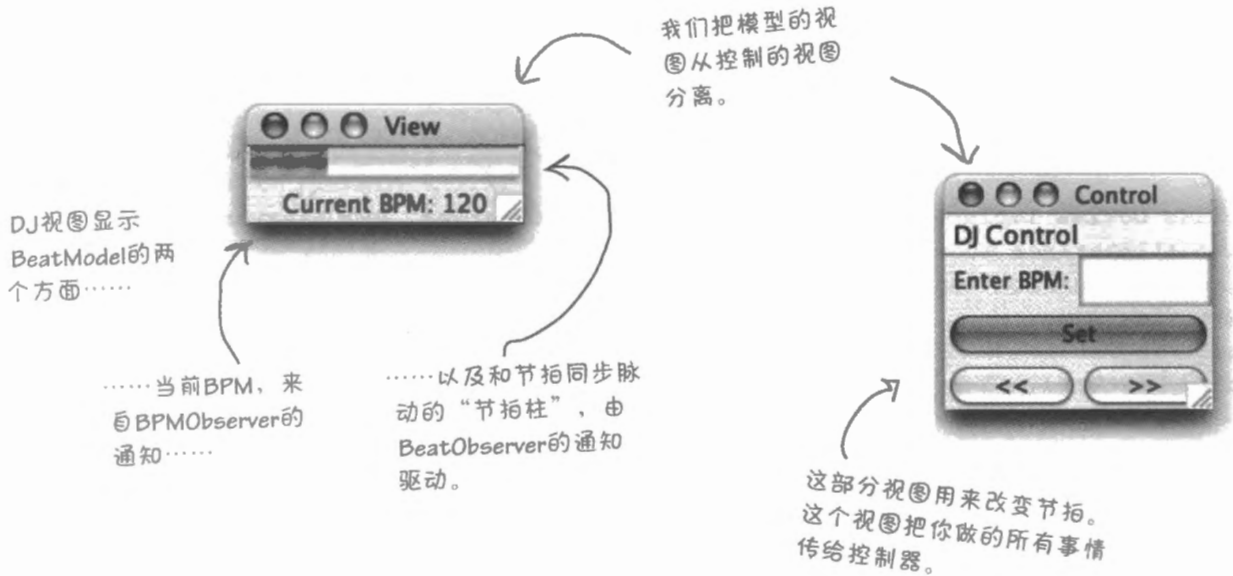
待烘焙代码

这个模型用一个音频片段来生成节拍。Java源文件里有所有DJ类的完整实现，可以在wickedlysmart.com网站取得，本章结尾也会列出代码。

视图

现在有趣的事情开始了，我们要把视图挂接上，使BeatModel可视化！

关于视图，要注意的第一件事是，我们的实现是要在两个分离的窗口显示。一个窗口包含当前BPM和脉动柱，另一个包含界面控制。为什么？因为我们要强调包含模型视图的界面和剩下的包含用户控制集的界面之间的差异。让我们详细看视图的这两个部分：



我们的BeatModel对视图没有做任何假设。这个模型是用观察者模式实现的，因此，当状态变化时，只是通知注册为观察者的任何视图。视图使用模型的API访问状态。我们实现了一种视图，你能想出可以在BeatModel中使用通知和状态的其他视图吗？

基于实时节拍的灯光秀。

一个基于BPM显示音乐种类（ambient、downbeat、techno等）的文本视图。

实现视图

视图的两个部分（模型的视图，用户界面控制的视图）显示在两个窗口中，但是属于同一个Java类。我们首先展示创建模型视图的代码，显示当前BPM和节拍柱。下一页我们展示创建用户界面控制的代码，显示BPM文本输入域和按钮。



注意!

这两页代码只是一个轮廓!

我们在这里将一个类分成两个部分，视图的一部分在本页展示，另一部分在下一页展示。实际上，所有代码属于同一个类——DJView.java。本章最后面将列出所有代码。

DJView是实时节拍和BPM变化的观察者。

```
public class DJView implements ActionListener, BeatObserver, BPMObserver {
    BeatModelInterface model;
    ControllerInterface controller;
    JFrame viewFrame;
    JPanel viewPanel;
    BeatBar beatBar;
    JLabel bpmOutputLabel;
```

视图持有模型和控制器的引用，控制器只由控制接口使用，一会就会看到……

这里，我们创建了几个用来显示的组件。

```
public DJView(ControllerInterface controller, BeatModelInterface model) {
    this.controller = controller;
    this.model = model;
    model.registerObserver((BeatObserver) this);
    model.registerObserver((BPMObserver) this);
}
```

构造器得到控制器和模型的引用，我们把引用存储在实例变量中。

```
public void createView() {
    // Create all Swing components here
}
```

我们也把这个注册为模型的BeatObserver和BPMObserver。

```
public void updateBPM() {
    int bpm = model.getBPM();
    if (bpm == 0) {
        bpmOutputLabel.setText("offline");
    } else {
        bpmOutputLabel.setText("Current BPM: " + model.getBPM());
    }
}
```

模型发生状态变化时，updateBPM()方法被调用。这时，我们更新当前BPM的显示。我们可以通过直接请求模型得到值。

```
public void updateBeat() {
    beatBar.setValue(100);
}
```

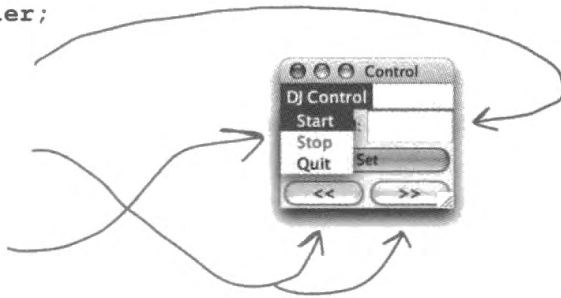
同样，当模型开始一个新的节拍时，updateBeat()方法被调用。这时，我们需要让脉动柱跳动。我们的做法是把脉动柱设为最大值(100)，让它处理脉动的动画。

继续实现视图……

现在，我们来看视图的用户界面控制部分的代码。这个视图通过告诉控制器做什么来让你控制模型，然后，控制器告诉模型做什么。记住，以下代码和其他的视图代码在同一个类文件中。

```
public class DJView implements ActionListener, BeatObserver, BPMObserver {
```

```
    BeatModelInterface model;
    ControllerInterface controller;
    JLabel bpmLabel;
    JTextField bpmTextField;
    JButton setBPMBButton;
    JButton increaseBPMBButton;
    JButton decreaseBPMBButton;
    JMenuBar menuBar;
    JMenu menu;
    JMenuItem startMenuItem;
    JMenuItem stopMenuItem;
```



```
    public void createControls() {
        // Create all Swing components here
    }
```

这个方法创建所有控件，并把它们放在界面上。它也会处理菜单。当stop或start项被选中时，会调用控制器的相应方法。

```
    public void enableStopMenuItem() {
        stopMenuItem.setEnabled(true);
    }
```

```
    public void disableStopMenuItem() {
        stopMenuItem.setEnabled(false);
    }
```

```
    public void enableStartMenuItem() {
        startMenuItem.setEnabled(true);
    }
```

```
    public void disableStartMenuItem() {
        startMenuItem.setEnabled(false);
    }
```

所有这些方法都允许菜单中的start和stop项变成enable或disable。一会我们会看到控制器使用这些方法来改变用户界面。

点击按钮时，这个方法被调用。

```
    public void actionPerformed(ActionEvent event) {
        if (event.getSource() == setBPMBButton) {
            int bpm = Integer.parseInt(bpmTextField.getText());
            controller.setBPM(bpm);
        } else if (event.getSource() == increaseBPMBButton) {
            controller.increaseBPM();
        } else if (event.getSource() == decreaseBPMBButton) {
            controller.decreaseBPM();
        }
    }
}
```

如果Set按钮被点击，会和新的BPM一起被传递到控制器。

类似的，如果递增或递减按钮被点击，这个信息会传给控制器。

现在来看控制器

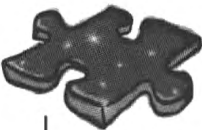
是时候编写丢失的片段了：控制器。别忘了，控制器是策略，我们把控制器插进视图，让视图变得聪明。

因为我们正在实现策略模式，所以需要从可能被插进DJ View的任何策略的接口开始。我们称此接口为ControllerInterface。

```
public interface ControllerInterface {  
    void start();  
    void stop();  
    void increaseBPM();  
    void decreaseBPM();  
    void setBPM(int bpm);  
}
```

这是所有视图能调用的
控制器方法。

在看过模型的接口后，你应该对这些方法感到熟悉。你可以停止和开始节拍生成，以及改变BPM。这个接口比BeatModel接口更“丰富”，因为你可以用“递增”和“递减”调整BPM。



设计谜题

你已经看到视图和控制器一起使用策略模式。你能画出这两个类的策略模式类图吗？

以下是控制器的实现：

控制器实现
ControllerInterface。

```
public class BeatController implements ControllerInterface {
```

```
    BeatModelInterface model;
```

```
    DJView view;
```

```
    public BeatController(BeatModelInterface model) {
```

```
        this.model = model;
```

```
        view = new DJView(this, model);
```

```
        view.createView();
```

```
        view.createControls();
```

```
        view.disableStopMenuItem();
```

```
        view.enableStartMenuItem();
```

```
        model.initialize();
```

```
    }
```

```
    public void start() {
```

```
        model.on();
```

```
        view.disableStartMenuItem();
```

```
        view.enableStopMenuItem();
```

```
    }
```

```
    public void stop() {
```

```
        model.off();
```

```
        view.disableStopMenuItem();
```

```
        view.enableStartMenuItem();
```

```
    }
```

```
    public void increaseBPM() {
```

```
        int bpm = model.getBPM();
```

```
        model.setBPM(bpm + 1);
```

```
    }
```

```
    public void decreaseBPM() {
```

```
        int bpm = model.getBPM();
```

```
        model.setBPM(bpm - 1);
```

```
    }
```

```
    public void setBPM(int bpm) {
```

```
        model.setBPM(bpm);
```

```
    }
```

```
}
```

控制器是MVC奥利奥曲奇饼中间的奶油，因此它同时和模型以及视图接触，把两者粘合起来。

模型传入控制器的构造器中，然后创建视图。

当你从用户界面菜单选择“Start”时，控制器调用模型的on()，然后改变用户界面，Start菜单项为disable，Stop菜单项为enable。

类似地，当你从菜单中选择“Stop”，控制器调用模型的off()，然后改变用户界面，Start菜单项为enable，Stop菜单项为disable。

如果递增按钮被点击，控制器就从模型取得当前BPM，加1，然后设置一个新BPM。

做法和上面一样，只是当前BPM值减1。

最后，如果用户界面被用来设置一个任意的BPM，控制器指示模型设置它的BPM。

注意：控制器在为视图做聪明的决定，视图只知道如何将菜单项变成开和关，不知道在什么情形下要disable。

全部结合在一起……

我们有了所需的一切：模型、视图和控制器。现在是时候把它们都放在一起了！我们会看到和听到它们怎样和谐地合作。

我们只需要一点点代码就能开始，代码不长：

```
public class DJTestDrive {  
  
    public static void main (String[] args) {  
        BeatModelInterface model = new BeatModel();  
        ControllerInterface controller = new BeatController(model);  
    }  
}
```

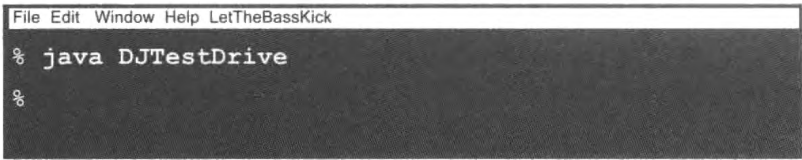
先创建一个模型……

……然后创建一个控制器，并把模型传送给它。记住，控制器创建视图，因此我们不必传送视图。



现在运行测试……

确认clip.wav文件在代码文件夹的最顶层。

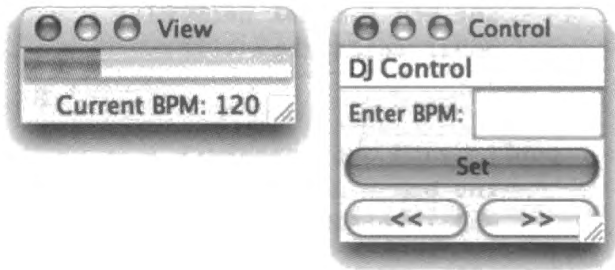


运行这个……

试试做这些

- 1 用Start菜单项启动节拍生成；注意到控制器随后把该项disable。
- 2 使用文本输入框以及递增和递减按钮改变BPM。注意视图显示如何反映变化，尽管实际上它没有逻辑链接到控件。
- 3 注意节拍柱如何一直保持正确的拍子，因为它是模型观察者。
- 4 播放你最喜欢的歌曲，看看你是否能用递增和递减控件来匹配节拍。
- 5 停止生成器。注意控制器如何disable Stop菜单项，并enable Start菜单项。

……你将看到这个。

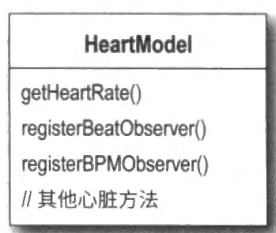


探索策略

让我们更进一步看策略模式，更好地感受它如何用在MVC中。我们也会看到另一个友善的模式出现，这个模式常常在MVC三人组的附近游荡：适配器模式。



想一下DJ视图做了什么，它显示了节拍速率以及脉动，这听起来是不是像某些其他东西？心跳？碰巧我们有一个心脏监视类，类图是这样的：



我们有一个方法，可以取得当前的心跳速率。

幸运的是，它的开发人员知道Beat和BPM观察者接口！



如果能够在HeartModel中复用我们当前的视图，这当然是件好事，但我们需要一个和这个模型合作的控制器。还有，HeartModel的接口不符合视图的期望，因为它的方法是getHeartRate()，而不是getBPM()。你将如何设计一些类，允许视图能和新的模型复用呢？简单写写你关于类设计的想法。

适配模型

一开始，我们希望把HeartModel适配成BeatModel。如果不这样做，视图就不能和模型合作，因为视图只知道如何getBPM()，而等价的心脏模型方法是getHeartRate()。要怎么做？我们打算使用适配器模式，当然了！实际上，适配器是MVC常用的技巧，使用适配器把模型适配到已有控制器和视图。

下面是将HeartModel适配到BeatModel的代码：

```
public class HeartAdapter implements BeatModelInterface {
    HeartModelInterface heart;
```

```
    public HeartAdapter(HeartModelInterface heart) {
        this.heart = heart;
    }
```

```
    public void initialize() {}
```

```
    public void on() {}
```

```
    public void off() {}
```

```
    public int getBPM() {
        return heart.getHeartRate();
    }
```

```
    public void setBPM(int bpm) {}
```

```
    public void registerObserver(BeatObserver o) {
        heart.registerObserver(o);
    }
```

```
    public void removeObserver(BeatObserver o) {
        heart.removeObserver(o);
    }
```

```
    public void registerObserver(BPMObserver o) {
        heart.registerObserver(o);
    }
```

```
    public void removeObserver(BPMObserver o) {
        heart.removeObserver(o);
    }
```

```
}
```

我们需要实现目标接口，在本例中就是BeatModelInterface。

在这里，我们存储心脏模型的引用。

我们不知道这些方法将对心脏做些什么，但是看起来很可怕，因此我们让它们“无操作”。

当getBPM()被调用，我们只是把它翻译成HeartModel的getHeartRate()调用。

我们不想对心脏做这种事！再一次，把它变成“无操作”。

这是我们的观察者方法。我们只是委托给所包装的心脏模型。

现在我们准备好写HeartController

有了HeartAdapter, 我们准备创建控制器, 并让视图和HeartModel一起运作。
这就是复用!

```
public class HeartController implements ControllerInterface {
    HeartModelInterface model;
    DJView view;

    public HeartController(HeartModelInterface model) {
        this.model = model;
        view = new DJView(this, new HeartAdapter(model));
        view.createView();
        view.createControls();
        view.disableStopMenuItem();
        view.disableStartMenuItem();
    }

    public void start() {}

    public void stop() {}

    public void increaseBPM() {}

    public void decreaseBPM() {}

    public void setBPM(int bpm) {}
}
```

就像BeatController一样, HeartController实现ControllerInterface。

和以前一样, 控制器创建视图, 并让所有东西粘合起来。

有一个变化: 我们传入一个HeartModel, 而不是BeatModel。

……在交给视图之前, 我们需要用适配器包装模型。

最后, HeartController把菜单项disable, 因为不需要它们。

这些方法没什么用; 毕竟, 我们不能像控制节拍机一样控制心跳。

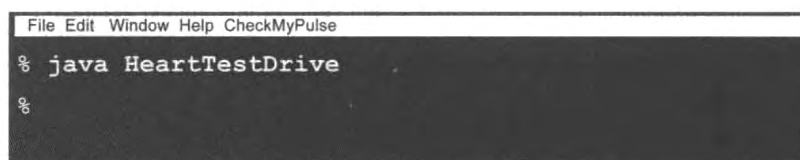
搞定! 是时候写一些测试代码了……

```
public class HeartTestDrive {

    public static void main (String[] args) {
        HeartModel heartModel = new HeartModel();
        ControllerInterface model = new HeartController(heartModel);
    }
}
```

我们所需要的只是创建一个控制器, 并传给它一个心脏监视器。

运行测试程序……



运行这个……

……会看到这个。



健康的心率。

试试这些：

- ① 注意，显示用在心跳上表现很好！节拍柱看起来就像脉搏。因为 HeartModel 也支持 BPM 观察者和 Beat 观察者，因此我们可以得到节拍的更新，就像 DJ 节拍一样。
- ② 因为心律有自然的变化，注意其显示是每分钟更新的。
- ③ 每次我们获取 BPM 的更新时，适配器把调用 `getBPM()` 翻译成调用 `getHeartRate()`。
- ④ 不能使用 Start 和 Stop 菜单项，因为控制器 disabled 了它们。
- ⑤ 其他按钮还可以用，只是没有效果，因为控制器对它们的实现是“无操作”。视图可以改变，以支持这些项的“无操作”。

没有蠢问题

问：看起来你真的是在断言一个事实：MVC中真的有组合模式。真的吗？

答：是的，组合模式真的在MVC中。不过，这的确是一个很好的问题。今天，像Swing这样的GUI包变得如此复杂，以至于我们很难注意它的内部结构，也很难注意到它在构造和更新显示时用了组合。甚至，当浏览器可以把标记语言转变成用户界面时，更难让我们看到其中的组合。

回到MVC刚刚被发现时，创建GUI需要许多手动干预，那时，模式作为MVC的一部分比现在更明显。

问：控制器会实现任何应用逻辑吗？

答：不会。控制器为视图实现行为，它聪明地将来自视图的动作翻译成模型上的动作。模型收到这些动作，实现应用逻辑来决定如何响应动作。控制器也要做一些决定，决定调用模型的哪个方法，但这不能算是“应用逻辑”。应用逻辑指管理和操纵存在于你的模型中的数据的代码。

问：我总是觉得“模型”这个词让我很头痛。我现在知道它是应用的内核，但是为什么要用这么模糊难懂的词汇来描述MVC的这个方面？

答：当给MVC起名时，他们需要用个M开头的词，否则就不能叫作MVC了。

说正经的，我们同意你的看法。一开始大家都会挠头，纳闷模型是什么。不过大家也都认识到，除了模型，还真找不到更好的词。

问：你说了许多关于模型的状态。这是不是意味着用到状态模式？

答：不，我们指的是一般意义上的状态。当然，有些模型使用状态模式来管理它们的内部状态。

问：我看过有些人把MVC的控制器描述成视图和模型之间的“中介者”（mediator）。控制器是否实现“中介者模式”？

答：我们还没有提到中介者模式（虽然你在本书的附录的模式概览中会看到），因此这里不会说太多细节，但中介者的意图是封装对象如何交互，并通过不让两个对象之间显式引用彼此，以促进松耦合。因此，从某种程度来看，控制器可以被看作一个中介者，因为视图不会直接设置模型的状态，而是通过控制器进行。但是记住，视图的确持有模型的引用，以访问模型的状态。如果控制器真的是中介者，视图就必须也要通过控制器来取得模型的状态。

问：视图总是必须向模型询问状态吗？我们是否可以用推送模型，在更新通知时，顺便发送模型状态？

答：是的，模型当然可以把状态和通知一起发送，我们可以对BeatModel做类似的事情，通过仅发送视图感兴趣的状态。如果你记得观察者模式一章，或许还记得这样做有什么缺点。如果不记得，翻回第2章，重新看一下。MVC模型已经被适配到许多类似的模型（尤其是Web浏览器/服务器环境），因此你会发现这条规则有一大堆例外。

问：如果有两个以上的视图，我是不是总是需要两个以上的控制器？

答：通常来说，运行时一个视图需要一个控制器，但是，同一个控制器类可以轻易管理多个视图。

问：视图不应该操纵模型；但是，我注意到你的实现中，对于改变模型状态的方法，视图拥有完全的访问权限。这危险吗？

答：你说得对。对于模型的方法集，我们给视图完全的权限。这么做的原因是为了保持简单，但可能会有某些情形，你想只让视图访问模型的部分API。有一个很棒的设计模式，允许你适配一个接口，起到只提供一个子集的作用，你能想起是哪个吗？



实际上我的大多数用户
界面是基于浏览器的。
以上知识对我有帮助吗？

是的！

MVC如此有用，以至于它已经适配到很多Web框架。当然，Web的运行方式和你的标准应用有差别，所以有不同的方案来使得MVC模式可以应用到Web。

Web应用有客户端（浏览器）和服务端。考虑到这一点，基于模型、视图和控制器驻留的位置，我们有不同的设计权衡。在瘦客户方案中，模型、大部分视图以及控制器都驻留在服务器，浏览器提供显示视图的方式，获取输入然后送到控制器。

另一个方案是单页面（single page）应用，几乎所有的模型、视图和控制器都驻留在客户端。这些是光谱的两端，你会发现框架的各种变化在于组件（即模型、视图和控制器）驻留在客户端还是服务端的偏重程度，还有杂化模型，一些组件在客户端和服务器之间共享。

有很多流行的MVC框架，像Spring Web MVC、Django、ASP.NET MVC、AngularJS、EmberJS、JavaScriptMVC、Backbone等。无疑还会有更多。每个框架的大多数部分都有其独特的方式来映射模型。现在你知道MVC模式了，就不会有任何问题，你可以将你的知识适配到选择的任何框架中。



设计工具箱内的工具

你的设计工具箱令所有人感到印象深刻。哇哦，看所有这些原则和模式，现在，还有复合模式！

OO原则

封装变化。

组合优于继承。

针对接口编程，不针对实现编程。

努力达到交互对象之间的松耦合设计。

类应该对扩展开放，对修改关闭。

依赖抽象，不要依赖具体类。

只和朋友交谈。

别打电话给我，我会打电话给你。

类应该只有一个变化的原因。

OO基础

抽象

封装

多态

继承

OO模式

代理模式，为另一个对象提供一个替身或占位符以控制对它的访问。

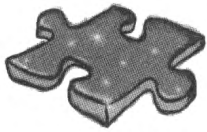
复合模式

复合模式把两个或更多的模式结合成一个解决方案，解决重复发生或一般的问题。

我们有一个新的类目！MVC是一个复合模式。

要点

- 模型-视图-控制器 (MVC) 是复合模式，结合了观察者、策略和组合模式。
- 模型使用观察者模式，以便可以保持观察者更新，同时依然保持两者之间解耦。
- 控制器是视图的策略，视图可以使用不同的控制器实现，得到不同的行为。
- 视图使用组合模式实现用户界面。用户界面通常组合了嵌套的组件，像面板、框架和按钮。
- 这些模式携手合作，解耦了MVC模型的三个层，这样可以保持设计干净又有弹性。
- 适配器模式用于把新的模型适配到已有的视图和控制器。
- MVC已经被适配到Web。
- 把MVC模式适配到各种客户/服务器的应用结构，就得到许多Web MVC框架。



习题解答



削尖你的铅笔 答案

QuackCounter也是一个Quackable。当我们为了扩展QuackObservable而改变Quackable时，我们不得不改变实现Quackable的每一个类，包括QuackCounter。

QuackCounter是Quackable，所以它现在也是QuackObservable。

```
public class QuackCounter implements Quackable {
```

```
    Quackable duck;
```

```
    static int numberOfQuacks;
```

```
    public QuackCounter(Quackable duck) {
```

```
        this.duck = duck;
```

```
    }
```

```
    public void quack() {
```

```
        duck.quack();
```

```
        numberOfQuacks++;
```

```
    }
```

```
    public static int getQuacks() {
```

```
        return numberOfQuacks;
```

```
    }
```

```
    public void registerObserver(Observer observer) {
```

```
        duck.registerObserver(observer);
```

```
    }
```

```
    public void notifyObservers() {
```

```
        duck.notifyObservers();
```

```
    }
```

```
}
```

这是一个QuackCounter正在装饰的鸭子。真正需要处理Observable方法的就是它。

这部分代码和之前的QuackCounter版本一样。

这是两个QuackObservable方法。注意，我们只是把两个调用委托给正在装饰的鸭子。



削尖你的铅笔

答案

要是呱呱叫学家想观察整个群怎么办？这意味着什么？像这样思考，如果我们观察一个组合，那么就是观察组合内的每个东西。因此，当你注册要观察某个群，群组合确定让你能观察所有的孩子，这可能还包括其他群。

```
public class Flock implements Quackable {
    List<Quackable> quackers = new ArrayList<Quackable>();
```

Flock是Quackable，所以现在它也是QuackObservable。

```
    public void add(Quackable duck) {
        ducks.add(duck);
    }
```

这里是Flock内的Quackable。

```
    public void quack() {
        Iterator<Quackable> iterator = quackers.iterator();
        while (iterator.hasNext()) {
            Quackable duck = iterator.next();
            duck.quack();
        }
    }
```

当你向Flock注册观察者时，其实是向Flock“内”的所有东西注册，即每一个Quackable，不管是一只鸭子还是另一个群。

```
    public void registerObserver(Observer observer) {
        Iterator<Quackable> iterator = ducks.iterator();
        while (iterator.hasNext()) {
            Quackable duck = iterator.next();
            duck.registerObserver(observer);
        }
    }

    public void notifyObservers() { }
```

我们遍历Flock内的所有Quackable，把调用委托给每个Quackable。如果Quackable是另一个Flock，做同样的事。

每个Quackable做自己的通知，这样，Flock就不必操心。当Flock将quack()委托给内部的每一个Quackable时，这就会发生。



削尖你的铅笔

答案

我们仍然依赖具体类来直接实例化鹅。你能够为鹅写一个抽象工厂吗？怎样创建“名鸭实鹅”？

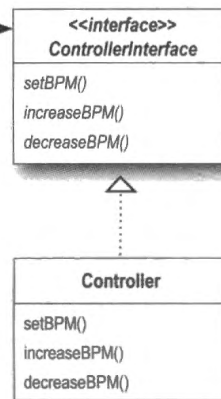
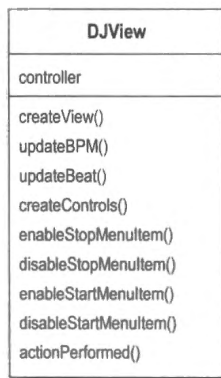
你可以给已有的鸭子工厂添加`createGooseDuck()`方法，或者你可以创建一个完全分离的工厂，来创建鹅的家族。



设计谜题解答

你已经看到视图和控制器一起使用策略模式。你能画出这两个类的策略模式类图吗？

视图把行为委托给控制器。所委托的行为的内容是关于如何基于用户输入控制模型。



具体控制器都必须实现 `ControllerInterface`。这就是策略接口。

我们可以插入不同的控制器来给视图提供不同的行为。



待烘焙 代码

以下是DJView的完整实现，展示了所有用于生成声音的MIDI代码，以及所有用于创建视图的Swing组件。你也可以到<https://www.wickedlysmart.com>下载这份代码。好好玩吧！

```
package headfirst.designpatterns.combined.djview;

public class DJTestDrive {

    public static void main (String[] args) {
        BeatModelInterface model = new BeatModel();
        ControllerInterface controller = new BeatController(model);
    }
}
```

节拍模型

```
package headfirst.designpatterns.combined.djview;

public interface BeatModelInterface {
    void initialize();

    void on();

    void off();

    void setBPM(int bpm);

    int getBPM();

    void registerObserver(BeatObserver o);

    void removeObserver(BeatObserver o);

    void registerObserver(BPMObserver o);

    void removeObserver(BPMObserver o);
}
```

```
package headfirst.designpatterns.combined.djview;

import java.util.*;
import javax.sound.sampled.AudioSystem;
import javax.sound.sampled.Clip;
import java.io.*;
import javax.sound.sampled.Line;

public class BeatModel implements BeatModelInterface, Runnable {
    List<BeatObserver> beatObservers = new ArrayList<BeatObserver>();
    List<BPMObserver> bpmObservers = new ArrayList<BPMObserver>();
    int bpm = 90;
    Thread thread;
    boolean stop = false;
    Clip clip;

    public void initialize() {
        try {
            File resource = new File("clap.wav");
            clip = (Clip) AudioSystem.getLine(new Line.Info(Clip.class));
            clip.open(AudioSystem.getAudioInputStream(resource));
        }
        catch(Exception ex) {
            System.out.println("Error: Can't load clip");
            System.out.println(ex);
        }
    }

    public void on() {
        bpm = 90;
        notifyBPMObservers();
        thread = new Thread(this);
        stop = false;
        thread.start();
    }

    public void off() {
        stopBeat();
        stop = true;
    }
}
```



待烘焙 代码

```

public void run() {
    while (!stop) {
        playBeat();
        notifyBeatObservers();
        try {
            Thread.sleep(60000/getBPM());
        } catch (Exception e) {}
    }
}

public void setBPM(int bpm) {
    this.bpm = bpm;
    notifyBPMObservers();
}

public int getBPM() {
    return bpm;
}

public void registerObserver(BeatObserver o) {
    beatObservers.add(o);
}

public void notifyBeatObservers() {
    for (int i = 0; i < beatObservers.size(); i++) {
        BeatObserver observer = (BeatObserver)beatObservers.get(i);
        observer.updateBeat();
    }
}

public void registerObserver(BPMObserver o) {
    bpmObservers.add(o);
}

public void notifyBPMObservers() {
    for (int i = 0; i < bpmObservers.size(); i++) {
        BPMObserver observer = (BPMObserver)bpmObservers.get(i);
        observer.updateBPM();
    }
}

```

```
public void removeObserver(BeatObserver o) {
    int i = beatObservers.indexOf(o);
    if (i >= 0) {
        beatObservers.remove(i);
    }
}

public void removeObserver(BPMObserver o) {
    int i = bpmObservers.indexOf(o);
    if (i >= 0) {
        bpmObservers.remove(i);
    }
}

public void playBeat() {
    clip.setFramePosition(0);
    clip.start();
}

public void stopBeat() {
    clip.setFramePosition(0);
    clip.stop();
}

}
```

视图

待烘焙
代码

```

package headfirst.designpatterns.combined.djview;

public interface BeatObserver {
    void updateBeat();
}

package headfirst.designpatterns.combined.djview;

public interface BPMObserver {
    void updateBPM();
}

package headfirst.designpatterns.combined.djview;

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class DJView implements ActionListener, BeatObserver, BPMObserver {
    BeatModelInterface model;
    ControllerInterface controller;
    JFrame viewFrame;
    JPanel viewPanel;
    BeatBar beatBar;
    JLabel bpmOutputLabel;
    JFrame controlFrame;
    JPanel controlPanel;
    JLabel bpmLabel;
    JTextField bpmTextField;
    JButton setBPMBButton;
    JButton increaseBPMBButton;
    JButton decreaseBPMBButton;
    JMenuBar menuBar;
    JMenu menu;
    JMenuItem startMenuItem;
    JMenuItem stopMenuItem;

    public DJView(ControllerInterface controller, BeatModelInterface model) {
        this.controller = controller;
        this.model = model;
        model.registerObserver((BeatObserver) this);
        model.registerObserver((BPMObserver) this);
    }
}

```

```

public void createView() {
    // 在这里创建所有的Swing组件
    viewPanel = new JPanel(new GridLayout(1, 2));
    viewFrame = new JFrame("View");
    viewFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    viewFrame.setSize(new Dimension(100, 80));
    bpmOutputLabel = new JLabel("offline", SwingConstants.CENTER);
    beatBar = new BeatBar();
    beatBar.setValue(0);
    JPanel bpmPanel = new JPanel(new GridLayout(2, 1));
    bpmPanel.add(beatBar);
    bpmPanel.add(bpmOutputLabel);
    viewPanel.add(bpmPanel);
    viewFrame.getContentPane().add(viewPanel, BorderLayout.CENTER);
    viewFrame.pack();
    viewFrame.setVisible(true);
}

public void createControls() {
    // 在这里创建所有的Swing组件
    JFrame.setDefaultLookAndFeelDecorated(true);
    controlFrame = new JFrame("Control");
    controlFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    controlFrame.setSize(new Dimension(100, 80));

    controlPanel = new JPanel(new GridLayout(1, 2));

    menuBar = new JMenuBar();
    menu = new JMenu("DJ Control");
    startMenuItem = new JMenuItem("Start");
    menu.add(startMenuItem);
    startMenuItem.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent event) {
            controller.start();
        }
    });
    stopMenuItem = new JMenuItem("Stop");
    menu.add(stopMenuItem);
    stopMenuItem.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent event) {
            controller.stop();
        }
    });
    JMenuItem exit = new JMenuItem("Quit");
    exit.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent event) {
            System.exit(0);
        }
    });
}

```



待烘焙 代码

```

menu.add(exit);
menuBar.add(menu);
controlFrame.setJMenuBar(menuBar);

bpmTextField = new JTextField(2);
bpmLabel = new JLabel("Enter BPM:", SwingConstants.RIGHT);
setBPMBButton = new JButton("Set");
setBPMBButton.setSize(new Dimension(10,40));
increaseBPMBButton = new JButton(">>");
decreaseBPMBButton = new JButton("<<");
setBPMBButton.addActionListener(this);
increaseBPMBButton.addActionListener(this);
decreaseBPMBButton.addActionListener(this);

JPanel buttonPanel = new JPanel(new GridLayout(1, 2));
buttonPanel.add(decreaseBPMBButton);
buttonPanel.add(increaseBPMBButton);

JPanel enterPanel = new JPanel(new GridLayout(1, 2));
enterPanel.add(bpmLabel);
enterPanel.add(bpmTextField);
JPanel insideControlPanel = new JPanel(new GridLayout(3, 1));
insideControlPanel.add(enterPanel);
insideControlPanel.add(setBPMBButton);
insideControlPanel.add(buttonPanel);
controlPanel.add(insideControlPanel);

bpmLabel.setBorder(BorderFactory.createEmptyBorder(5,5,5,5));
bpmOutputLabel.setBorder(BorderFactory.createEmptyBorder(5,5,5,5));

controlFrame.getRootPane().setDefaultButton(setBPMBButton);
controlFrame.getContentPane().add(controlPanel, BorderLayout.CENTER);

controlFrame.pack();
controlFrame.setVisible(true);
}

public void enableStopMenuItem() {
    stopMenuItem.setEnabled(true);
}

public void disableStopMenuItem() {
    stopMenuItem.setEnabled(false);
}
}

```

```
public void enableStartMenuItem() {
    startMenuItem.setEnabled(true);
}

public void disableStartMenuItem() {
    startMenuItem.setEnabled(false);
}

public void actionPerformed(ActionEvent event) {
    if (event.getSource() == setBPMBButton) {
        int bpm = 90;
        String bpmText = bpmTextField.getText();
        if (bpmText == null || bpmText.contentEquals("")) {
            bpm = 90;
        } else {
            bpm = Integer.parseInt(bpmTextField.getText());
        }
        controller.setBPM(bpm);
    } else if (event.getSource() == increaseBPMBButton) {
        controller.increaseBPM();
    } else if (event.getSource() == decreaseBPMBButton) {
        controller.decreaseBPM();
    }
}

public void updateBPM() {
    int bpm = model.getBPM();
    if (bpm == 0) {
        bpmOutputLabel.setText("offline");
    } else {
        bpmOutputLabel.setText("Current BPM: " + model.getBPM());
    }
}

public void updateBeat() {
    beatBar.setValue(100);
}
}
```

控制器

```
package headfirst.designpatterns.combined.djview;
```

```
public interface ControllerInterface {
    void start();
    void stop();
    void increaseBPM();
    void decreaseBPM();
    void setBPM(int bpm);
}
```

待烘焙
代码

```
package headfirst.designpatterns.combined.djview;

public class BeatController implements ControllerInterface {
    BeatModelInterface model;
    DJView view;

    public BeatController(BeatModelInterface model) {
        this.model = model;
        view = new DJView(this, model);
        view.createView();
        view.createControls();
        view.disableStopMenuItem();
        view.enableStartMenuItem();
        model.initialize();
    }

    public void start() {
        model.on();
        view.disableStartMenuItem();
        view.enableStopMenuItem();
    }

    public void stop() {
        model.off();
        view.disableStopMenuItem();
        view.enableStartMenuItem();
    }

    public void increaseBPM() {
        int bpm = model.getBPM();
        model.setBPM(bpm + 1);
    }

    public void decreaseBPM() {
        int bpm = model.getBPM();
        model.setBPM(bpm - 1);
    }

    public void setBPM(int bpm) {
        model.setBPM(bpm);
    }
}
```

心脏模型

```
package headfirst.designpatterns.combined.djview;

public class HeartTestDrive {

    public static void main (String[] args) {
        HeartModel heartModel = new HeartModel();
        ControllerInterface model = new HeartController(heartModel);
    }
}

package headfirst.designpatterns.combined.djview;

public interface HeartModelInterface {
    int getHeartRate();
    void registerObserver(BeatObserver o);
    void removeObserver(BeatObserver o);
    void registerObserver(BPMObserver o);
    void removeObserver(BPMObserver o);
}

package headfirst.designpatterns.combined.djview;

import java.util.*;

public class HeartModel implements HeartModelInterface, Runnable {
    List<BeatObserver> beatObservers = new ArrayList<BeatObserver>();
    List<BPMObserver> bpmObservers = new ArrayList<BPMObserver>();
    int time = 1000;
    int bpm = 90;
    Random random = new Random(System.currentTimeMillis());
    Thread thread;

    public HeartModel() {
        thread = new Thread(this);
        thread.start();
    }

    public void run() {
        int lastrate = -1;

        for(;;) {
            int change = random.nextInt(10);
            if (random.nextInt(2) == 0) {
                change = 0 - change;
            }
            int rate = 60000/(time + change);
```



待烘焙
代码

```

        if (rate < 120 && rate > 50) {
            time += change;
            notifyBeatObservers();
            if (rate != lastrate) {
                lastrate = rate;
                notifyBPMObservers();
            }
        }
        try {
            Thread.sleep(time);
        } catch (Exception e) {}
    }
}

public int getHeartRate() {
    return 60000/time;
}

public void registerObserver(BeatObserver o) {
    beatObservers.add(o);
}

public void removeObserver(BeatObserver o) {
    int i = beatObservers.indexOf(o);
    if (i >= 0) {
        beatObservers.remove(i);
    }
}

public void notifyBeatObservers() {
    for(int i = 0; i < beatObservers.size(); i++) {
        BeatObserver observer = (BeatObserver)beatObservers.get(i);
        observer.updateBeat();
    }
}

public void registerObserver(BPMObserver o) {
    bpmObservers.add(o);
}

public void removeObserver(BPMObserver o) {
    int i = bpmObservers.indexOf(o);
    if (i >= 0) {
        bpmObservers.remove(i);
    }
}

public void notifyBPMObservers() {
    for(int i = 0; i < bpmObservers.size(); i++) {
        BPMObserver observer = (BPMObserver)bpmObservers.get(i);
        observer.updateBPM();
    }
}
}
}

```

心脏适配器

```
package headfirst.designpatterns.combined.djview;

public class HeartAdapter implements BeatModelInterface {
    HeartModelInterface heart;

    public HeartAdapter(HeartModelInterface heart) {
        this.heart = heart;
    }

    public void initialize() {}

    public void on() {}

    public void off() {}

    public int getBPM() {
        return heart.getHeartRate();
    }

    public void setBPM(int bpm) {}

    public void registerObserver(BeatObserver o) {
        heart.registerObserver(o);
    }

    public void removeObserver(BeatObserver o) {
        heart.removeObserver(o);
    }

    public void registerObserver(BPMObserver o) {
        heart.registerObserver(o);
    }

    public void removeObserver(BPMObserver o) {
        heart.removeObserver(o);
    }
}
```

控制器

待烘焙
代码

```
package headfirst.designpatterns.combined.djview;

public class HeartController implements ControllerInterface {
    HeartModelInterface model;
    DJView view;

    public HeartController(HeartModelInterface model) {
        this.model = model;
        view = new DJView(this, new HeartAdapter(model));
        view.createView();
        view.createControls();
        view.disableStopMenuItem();
        view.disableStartMenuItem();
    }

    public void start() {}

    public void stop() {}

    public void increaseBPM() {}

    public void decreaseBPM() {}

    public void setBPM(int bpm) {}
}
```

